



UnionPay Integrated Circuit Card Specifications — Auxiliary Specifications

Part IV Debit/Credit Application Issuer's Security Guide

Version 2014

THIS PAGE IS INTENTIONALLY LEFT BLANK.

Table of Contents

SUMMARY OF REVISIONS	VIII
1 APPLICATION SCOPE	1
2 NORMATIVE REFERENCES	2
3 TERMINOLOGY AND DEFINITIONS	3
3.1 UNIONPAY INTEGRATED CIRCUIT (IC) CARD SPECIFICATIONS	3
3.2 FINANCIAL IC CARD DEBIT/CREDIT APPLICATIONS ROOT CA	3
3.3 AUTHENTICATION	3
3.4 PUBLIC KEY CERTIFICATES	3
3.5 CERTIFICATION AUTHORITY	3
3.6 CRYPTOGRAPHIC ALGORITHMS	3
3.7 HASH FUNCTIONS	4
3.8 CRYPTOPERIOD	4
3.9 DIGITAL SIGNATURES	4
3.10 DUAL CONTROLS	4
3.11 HASH VALUES	4
3.12 INTEGRATED CIRCUIT CARD (IC CARD)	4
3.13 KEY COMPONENTS	5
3.14 KEY PAIRS	5
3.15 KEY MATERIAL	5
3.16 PAYMENT SYSTEMS	5
3.17 PHYSICALLY SECURE DEVICES	5

3.18 PRIVATE KEY	5
3.19 PSEUDO-RANDOM	5
3.20 PUBLIC KEYS	5
3.21 SECRET KEY	6
3.22 SECURE CRYPTOGRAPHIC DEVICE.....	6
3.23 SIGNATURE KEY.....	6
3.24 SPLIT KNOWLEDGE	6
3.25 VERIFICATION KEYS	6
4 SYMBOLS AND ABBREVIATIONS	7
5 UICS STANDARD CREDIT APPLICATION AND CIPHERTEXT TECHNOLOGY OVERVIEW	10
5.1 INTRODUCTION	10
5.2 PAYMENT SYSTEM MODEL	10
5.2.1 Cardholders	10
5.2.2 Merchants	10
5.2.3 Issuers.....	11
5.2.4 Acquirers.....	11
5.2.5 Payment System	12
5.3 FOUNDATION OF CIPHERTEXT TECHNOLOGY	12
5.3.1 Symmetrical Algorithms	13
5.3.2 Asymmetrical Algorithms	14
5.4 UICS CREDIT APPLICATION VERIFICATION METHODS.....	17
5.4.1 Card and Data Authentication Methods.....	17

5.4.2	Static Data Authentication (SDA).....	19
5.4.3	Dynamic Data Authentication (DDA/CDA).....	19
5.4.4	Comparison of Authentication Methods	20
5.4.5	Cardholder Authentication Methods	22
5.5	TRANSACTION AUTHORIZATION SYSTEM	23
6	UICS CARD ISSUER SECURITY	24
6.1	ISSUER TASKS	24
6.1.1	Card Issuance Preparation	25
6.1.2	Secure Counting Devices.....	29
6.1.3	Card Production (SDA).....	30
6.1.4	Card Production (DDA and CDA).....	31
6.1.5	Card Issuance	32
6.2	POST-ISSUANCE CONDUCT OF ISSUER	32
6.2.1	Transaction Clearing	33
6.2.2	Key Invalidation.....	33
7	KEY MANAGEMENT PRACTICES	35
7.1	KEY PRODUCTION - GENERAL GUIDE	35
7.1.1	RSA Key Generation	38
7.1.2	Certificate Generation	38
7.1.3	Symmetrical Key Generation and Dispersal	38
7.1.4	Random Number Generators Used for Key Generation	39
7.2	KEY STORAGE AND TRANSMISSION	39

7.2.1	Key Storage.....	39
7.2.2	RSA Key Storage and Transmission.....	41
7.2.3	3DES Key Transmission and Storage	42
7.3	KEY MANAGEMENT PRACTICES AND RESPONSIBILITIES	42
7.3.1	Key Management Personnel Planning	42
7.3.2	Key Management Functions.....	43
7.3.3	Process Used When Delivering Key Materials to Third-Party Institutions.....	44
7.3.4	Physical Key Security Protection Process of Third-Parties.....	44
8	HARDWARE AND SOFTWARE SECURITY CONSIDERATIONS	46
8.1	SECURE CRYPTOGRAPHIC DEVICES (SCD).....	46
8.1.1	Anti-Tamper Requirements.....	46
8.1.2	Key Storage – General Guide.....	48
8.2	IC CARD APPLICATION SECURITY	50
8.3	FRAUD DETECTION	51
9	UICS DEBIT/CREDIT APPLICATION KEYS	53
9.1	UICS SPECIFICATION DEBIT/CREDIT APPLICATION KEYS	53
9.1.1	3DES Main Keys	53
9.1.2	Card Static Data Authentication (SDA) Keys	54
9.1.3	Card Dynamic Data Authentication (DDA) Keys.....	55
9.1.4	Other Keys	55
9.2	EXPORTING PROCEDURE KEYS.....	55
9.3	KEY LENGTH AND USAGE CYCLE.....	56

9.4 UICS DEBIT/CREDIT APPLICATION KEY SUMMARY	58
10 UICS DEBIT/CREDIT APPLICATION KEY CRYPTOGRAPHIC ALGORITHMS.....	61
10.1 CARD MAIN KEY AND PROCESS KEY GENERATION CRYPTOGRAPHIC ALGORITHMS	61
10.1.1 Card Main Key Generation Cryptographic Algorithms.....	61
10.1.2 Card Main Key Process Key Generation Cryptographic Algorithms	62
10.2 CARD AND ISSUER ONLINE APPLICATION CIPHERTEXT PROCESSING	63
10.2.1 Online Application Ciphertext Generation Cryptographic Algorithms	63
10.2.2 Online Application Ciphertext Generation.....	64
10.3 CARD AND ISSUER SECURE MESSAGE PROCESSING	66
10.3.1 Secure Message Process Keys.....	66
10.3.2 Secure Message Cryptographic Algorithm.....	66
10.4 OFFLINE DATA AUTHENTICATION PROCESSES.....	71
10.4.1 Financial IC Card Debit/Credit Application Public Key Authentication System Overview.....	71
10.4.2 Offline Authentication Process Keys	73
10.4.3 Offline Authentication Process Cryptographic Algorithms	74
10.4.4 Offline Authentication Generation and Verification Processes.....	74
10.5 IC CARD CA PUBLIC KEY FILES.....	80
10.6 ISSUER PUBLIC KEY INPUT FILES	80
10.7 ISSUER PUBLIC KEY OUTPUT FILES	80
10.8 CARD CERTIFICATES AND SIGNED STATIC DATA GENERATION	80

10.8.1 Issuer-Issued Card Certificates.....	80
10.8.2 Issuer CA-Issued Card Static Signatures.....	86
10.9 CARD PERSONALIZATION PROCEDURES KEYS AND CRYPTOGRAPHIC ALGORITHMS	89
10.9.1 Card Personalization Procedure Security Regulations	89
10.9.2 Card Personalization Keys.....	90
11 UICS DEBIT/CREDIT APPLICATION TRANSACTION CRYPTOGRAPHIC OPERATIONS.....	95
11.1 ONLINE-AUTHORIZED CARD TRANSACTIONS:	96
11.1.1 Export of Main Card Keys.....	96
11.1.2 Card AV Process Key Export.....	97
11.1.3 ARQC Generation	98
11.1.4 ARPC Generation	99
11.1.5 TC Generation	100
11.2 PIN CHANGES.....	101
11.2.1 Script Encryption Card Main Key Export	101
11.2.2 Script Encryption Process Key Export	102
11.2.3 Script Encryption	103
11.2.4 Script MAC Card Main Key Export.....	103
11.2.5 Script MAC Process Key	104
11.2.6 Pin Change Script MAC Calculations.....	105
11.3 STATIC DATA AUTHENTICATION.....	106
11.3.1 Verification of Signed Static Application Data	106

11.4 DYNAMIC DATA AUTHENTICATION	109
11.4.1 <i>Generation of Dynamic Application Data Signature</i>	110
11.4.2 <i>IC Card Public Key Certificate Verification</i>	113
11.4.3 <i>Signed Dynamic Application Data Verification</i>	118
11.5 CDA GENERATION AND VERIFICATION	120
11.5.1 <i>Composite DDA/application ciphertext generation (CDA)</i>	120
11.5.2 <i>CDA Verification</i>	123

Summary of Revisions

The change listed below is associated with the current version.

Description of Change	Where to look
Revised: Changed Nic from “no smaller than 768 bits” to “no less than 1024 bits”.	9.3
Revised: Changed PBOC 2.0 to UICS.	Whole document
Revised: Changed “Financial Integrated Circuit (IC) Card Specifications” to “UnionPay Integrated Circuit (IC) Card Specifications”, and revised the concept.	3.1
Removed: Removed Table 3 and corresponding descriptions within Table 3.	9.3
Revised: Changed duplicate specification to reference.	10.5
Revised: Changed duplicate specification to reference.	10.6
Revised: Changed duplicate specification to reference.	10.7

1 Application Scope

This book applies to all UPI Participants.

2 Normative References

These standards reference the provisions of the following documents, and these provisions may therefore be considered to be provisions of the standards themselves. For all dated reference documents, all later amendments (except errata corrections) or modifications are not necessarily applicable to these standards. For this reason it is recommended that all parties associated with this agreement conduct all necessary research and come to an understanding regarding what newer versions of these documents remain applicable. For all non-dated documents, their most recent version may be considered to be applicable to these standards.

UnionPay Integrated Circuit Card Specifications – Basic Specifications

*UnionPay Integrated Circuit Cards Specifications – Auxiliary Specifications
Part 2&3*

*EMV 4.3 Integrated Circuit Card Specifications for Payment Systems, Books 1
to 4*

3 Terminology and Definitions

These standards use the following terminology and definitions:

3.1 UnionPay Integrated Circuit (IC) Card Specifications

One of UnionPay enterprise standards, first issued and published on November, 2012, include 3 volumes: Basic Specifications, Product Specifications and Auxiliary Specifications, totally 25 parts. This standard only concerns debit/credit applications, and hereafter will simply be referred to as “UICS”.

3.2 Financial IC Card Debit/Credit Applications Root CA

Created with the under the unified management of UnionPay, and servicing the Root Authentication Centre for IC card security applications within the financial industry. Hereafter referred simply as “IC Card Root CA”. The application system that implements the functionality of the Root Authentication System is the “Financial IC Card Debit/Credit Application Root CA System”. Hereafter this will simply be referred to as the “IC Card Root CA System”.

3.3 Authentication

An identification process used to confirm the authenticity and integrity of data.

3.4 Public Key Certificates

Public key certificates used by the Authentication Centre are created with their private key signatures, which are a combination of public keys, physical identification, and other forms of information. These certificates cannot be tampered with or altered.

3.5 Certification Authority

Several trusted organizational entities that generate and issue public key certificates for other entities.

3.6 Cryptographic Algorithms

Rules and processes such as encryption and decryption that are used to authenticate or protect data. It is impossible to determine any related encryption control parameters for these algorithms—such as keys or private keys—except by exhaustive search.

3.7 Hash Functions

Maps data from a small space to a smaller one; these hash functions have the following characteristics:

1. Computationally speaking, it is impossible to discern an input for a given hash function based solely on that hash function's known output alone.
2. Computationally speaking, it is impossible to generate a second input for a known hash function output that would be capable of recreating said output.

3.8 Cryptoperiod

A set period of time during which either a set key will be authorized for usage or a key for a given system will be effective.

3.9 Digital Signatures

A cryptographic transformation of data that if done correctly will be capable of the following functions:

- Authentication of the source of the data
- Protection of the integrity of the data
- Lack of signer deniability

3.10 Dual Controls

The usage of two or more entities to protect sensitive information or functions such that no single entity can access or use said information or functions.

3.11 Hash Values

The result obtained when a hash function is used on a given piece of information.

3.12 Integrated Circuit Card (IC Card)

A card packed internally with one or more integrated circuits that can be used for processing and storage.

3.13 Key Components

A key composed of at least one or two random parameters, whereupon those parameters are of one or more different types.

3.14 Key Pairs

Refers to a public key and the private key that corresponds to it.

3.15 Key Material

The data used to build and sustain a key (such as: key, certificate, initial vector, etc.)

3.16 Payment Systems

A payment system includes a requisite number of parties; here, an issuer and acquirer entrust certain responsibilities to other entities in accordance with system rules and associated risk assessments.

3.17 Physically Secure Devices

A module that has an ignorable level of risk in regards to the possibility of undetected and unauthorized access and modification of its data contents.

3.18 Private Key

Within an Asymmetrical cryptographic algorithmic (public key) system, entity key pairs are only known to that entity. The private key discussed here is different from the encrypted secret key type found in symmetrical cryptographic algorithms.

3.19 Pseudo-random

Within a given system, the output of that system is, statistically speaking, random and unpredictable, despite the fact that the output is generated by a deterministic algorithm process.

3.20 Public Keys

The entity key within an Asymmetrical key system that can be known publically.

3.21 Secret Key

Used for symmetrical cryptographic algorithmic keys; should this key become publically known, then the encryption of the entire symmetrical algorithm system will be compromised. Secret keys are different from the private keys of Asymmetrical key pairs.

3.22 Secure Cryptographic Device

Provides secure storage for secret information such as keys and uses this secret information to provide security services.

3.23 Signature Key

See “Secret Keys”

3.24 Split Knowledge

A situation in which two or more entities manage a key in such a way that each party controls some piece of information required to form that key; these “key managers” can then cooperate with each other to either create or reactivate this key. However, no single key manager is capable of doing so by itself without the cooperation of the others.

3.25 Verification Keys

See “Public Keys”

4 Symbols and Abbreviations

The following symbols and abbreviations are applicable to these standards:

AAC	Application Authentication Cryptogram
AC	Application Cryptogram
AIP	Application Interchange Profile
ARPC	Authorization Response Cryptogram
ARQC	Authorization Request Cryptogram
ATC	Application Transaction Counter
ATM	Automated Teller Machine
CA	Certificate Authority
CDA	Combined DDA/AC Generation
CDOL	Card Risk Management Data Object List
CVM	Cardholder Verification Method
DDA	Dynamic Data Authentication
DES	Data Encryption Standard
EMV	Europay MasterCard VISA
HSM	Hardware Security Module
IAD	Issuer Application Data
IC	Integrated Circuit

IC Card	Integrated Circuit Card
IMK _{AC}	Issuer Master Key for Application Cryptogram
IMK _{SMC}	Issuer Master Key for Secure Messaging for Confidentiality
IMK _{SMI}	Issuer Master Key for Secure Messaging for Integrity
MAC	Message Authentication Cyphertext
MK	Master Key
MK _{AC}	Card Master Key for Application Cryptogram Computation
MK _{SMC}	Card Master Key for Secure Messaging for Confidentiality
MK _{SMI}	Card Master Key for Secure Messaging for Integrity
PAN	Application Primary Account Number
PDOL	Processing Options Data Object List
POS	Point of Sale
PBOC	People's Bank of China
RSA	Rivest, Shamir , and Adleman Algorithm
SCD	Secure Cryptographic Device
SDA	Session Key for Application Cryptogram Computation
SK _{AC}	Session Key for Application Cryptogram Computation
SK _{SMC}	Session Key for Secure Messaging for Confidentiality
SK _{SMI}	Session Key for Secure Messaging for Integrity

TC	Transaction Certificate
3DES	Two Key Triple DES
TVR	Terminal Verification Results
UICS	UnionPay IC Card Specifications

5 UICS Standard Credit Application and Ciphertext Technology Overview

5.1 Introduction

The purpose of this overview is to provide a framework introduction for the issuer security guide while at the same time introducing the UICS UICS Debit/Credit Application Payment System model and the roles played by the various entities within its framework. Additionally it will also introduce the cryptographic foundations of the payment system.

5.2 Payment System Model

The UICS Debit/Credit Application Payment System includes the following types of entities:

- Cardholders,
- Merchants,
- Issuers,
- Acquirers,
- Payment System Organizations (UnionPay)

The roles of these payment system entities are as follows:

5.2.1 Cardholders

The roles of cardholders are as follows:

- Enters into a signed agreement with an issuer and thereby receives a UnionPay IC card, a UICS debit/credit payment product.
- Selects, records, or (possibly) changes the cardholder PIN.
- Inserts the UnionPay IC Card (or allows it to be inserted by a cashier) into a UnionPay IC Card UICS Debit/Credit Application Payment Terminal (such as an ATM, POS, etc.)

5.2.2 Merchants

Merchant roles include:

- Enters into a signed agreement with an acquirer such that the merchant agrees to allow its terminals to accept the acquirer's IC Card products and payment methods; the merchant will then offer acquirer services to IC Cards.
- Agrees to process transactions using payment product IC Cards.
- Utilizes terminals to collect purchase transaction information and then send that information to the issuer in exchange for purchase funds.

5.2.3 Issuers

Issuer roles include:

- Enters into a signed agreement with the cardholder to complete the personalization of a UICS Debit/Credit Payment Application-capable UnionPay IC Card and then issue that card to the cardholder. This process includes card generation and the installation of all required keys and payment applications.
- Processes online transactions. This includes authentication of the information and ciphertext messages transmitted by the card, the generation of cyphertext messages, and all necessary authentication procedures between the card and the issuer. At the same time, during the transaction approval process the issuer also conducts verification of the cardholder's online PIN.
- Generates new card application scripts if necessary.
- Transmits funds to acquirer for card transactions.
- Transmits keys as necessary to other entities so as to maintain the normal operation of the payment system.

5.2.4 Acquirers

The roles of acquirers are as follows:

- Enters into a written agreement with merchants and deploys that merchant's payment terminals. This includes the installation and management of IC Card Root CA public keys inside certain terminals and guaranteeing the completeness of the IC Card public key information within those terminals.

- Process card payment transactions and transmit payment transaction funds to merchants.
- Transmit collected transaction data to issuers and receive transaction settlements in return.
- Sets the receiving requirements for offline/online transactions and manages all associated risks.

5.2.5 Payment System

The roles of the payment system are as follows:

- Develops system rules for products and services and verifies that these rules are being followed.
- Generates and issues IC Card Root CA public keys.
- Publish UICS Debit/Credit application IC Card public key certificates for the issuer.
- Operates all payment communication networks necessary between issuer and acquirer for all UICS card transactions.
- Processes transaction settlements on the payment network described above.

5.3 Foundation of Ciphertext Technology

Cyphertext technology is used heavily in within IC Card applications. Historically, cyphertext technology is used to encrypt information. Aside from information encryption, modern encryption technology also includes other encryption services such as data integrity, authentication, and non-repudiation. As these different types of encryption services increases, the importance of control technologies for different kinds of encryption products and services for different clients grows every day. This section will provide a general description of these technical standards.

The key management principles established below are applicable to the keys used during the card personalization process, authentication protocols, transaction settlements, and other similar environments. These principles are based on encryption technology standards.

Current encryption technology is based on two components: cryptographic algorithms and keys. Cryptographic algorithms define how to create an encrypted document from a non-encrypted one (or the reverse, if it is an encryption algorithm), define how to obtain digital signatures from data, and (if it is a digital signature algorithm) how to verify digital signatures. The input for cryptographic algorithms is typically a key made up of a sequence of bits, which guarantees that even an unauthorized person with knowledge of the algorithm is unable to either decrypt and access sensitive information or pass themselves off as someone else.

Assuming a publically known algorithm, the security of modern encryption technology is dependent on the integrity and secure management of keys. Cryptological algorithms are typically released publically and it is common for cryptography professionals to conduct extensive research on them. For instance, the DES algorithm has already been released in several different standards and other document forms while RSA depends on well-known mathematical principles. The security of these two different cryptographic algorithms is completely dependent on how keys and private keys are managed during their lifecycle. It is in this way that each user key protects against the unauthorized decryption of sensitive data and digital signature forgery.

Generally speaking there are two kinds of cryptographic algorithms: symmetrical (or secret) key algorithms and Asymmetrical (or public/private key) cryptographic algorithms. Both symmetrical and Asymmetrical algorithms apply to UICS specification debit/credit applications.

5.3.1 Symmetrical Algorithms

Symmetrical (or secret) key algorithms require that the same secret key be used during both the encryption and decryption processes. In this case the security of the algorithm completely depends on the security of the secret key.

UICS debit/credit applications support the usage of the DES algorithm, widely used today in both the financial and industrial sectors. The DES algorithm is a type of “block cipher”, as this kind of algorithm processes data as separate pieces. The input for a DES algorithm is a 64-bit data set; this data set is processed with a 56-bit key in a 16-step iterative operation to produce a 64-bit data set output.

The 3DES algorithm actually uses two 56-bit DES keys and then simply repeats the DES algorithm three times. In this process, the first DES key is used to convert data using the DES algorithm. The output of this first calculation is then converted by the DES algorithm again using a second DES key; the output of

this second calculation is then converted by the DES algorithm a third time using the first DES key. In this document a DES key actually refers to a pair of DES keys, and the entire two-key operation is referred to as either 3DES or Double Key Length 3DES. All DES key lengths used in UICS specification debit/credit applications are 16 byte/128-bit, including the 16-bit verification digit.

The potential risks of DES keys include:

- Physical leakage of DES secret keys.
- Bypass attacks conducted on the DES key stored inside the internal memory of an IC Card.
- Exhaustive search attacks conducted on DES keys; computationally speaking, it is currently impossible to perform this kind of attack on a two-key 3DES system.
- Actual breakage of the DES algorithm; computationally speaking, it is currently impossible to break the 3DES algorithm.

UICS specifications utilize 3DES symmetrical cryptographic algorithms to process online transaction messages to completely protect data integrity and conducts online authorization of all entities (e.g. both card and issuer). Within a personalized IC card system the 3DES algorithm is also used to encrypt and protect the integrity of all transmitted data.

Within UICS specification debit/credit applications, block cipher algorithms referring to symmetrical cryptographic algorithms are usually based on simple logic operations that are iterated many times. For instance, in DES encryption these logic operations are iterated 16 times, making it a 16-cycle operation. These types of algorithms can be completed using very little resources and in very short periods of time, making them useful for encrypting and protecting the integrity of large sets of data. There are currently no known exhaustive attack methods that can be used to conduct analysis and defeat of 3DES encryptions. However, other methods--such as bypass attacks--do exist.

5.3.2 Asymmetrical Algorithms

Asymmetrical algorithms—or public key algorithms—require that both ends of transaction use two different but connected keys; a public key and a private key. Public keys are made known through credible channels while private keys are securely managed only by the private key's user. Some public key algorithms

such as RSA can be used for both encryption operations and digital signature operations. During a public key algorithm encryption operation, the encryptor will communicate to a user that a public key has been used to encrypt a piece of data and then transmit the result of that encryption. Once the user has received this encrypted data it will then decrypt it using a private key. RSA can also be used for digital signatures; the signer will use a private key to conduct a RSA operation on the hash value of a given piece of data; any other user who receives this data can then use a public key to conduct an RSA operation to determine the hash value of that user's digital signature; if the calculated hash value is the same as the hash value received from the user then the signature has been verified. Digital signatures derived from the RSA asymmetrical algorithm are used by UICS specification debit/credit applications for offline card verification processes.

Public key algorithms are typically based on a difficult mathematical problem, the design of which guarantees that there does not exist a superior real solution which could be used to defeat the algorithm. RSA algorithms are based on sum factorization due to the fact that it is not currently computationally feasible to find what two prime numbers multiplied together created a given sum if only the sum itself is known. In RSA, this sum is the public key modulus, and the factors of this modulus are RSA private keys. The larger the RSA public key modulus (i.e. modulus/public key length) the more difficult it is to factor and determine the corresponding private keys.

5.3.2.1 Asymmetrical (RSA) Keys

The security of an RSA algorithm-derived private key is dependent on the following factors:

- Length of the RSA public key modulus as represented by bit number, such as 1024-bit, 1152-bit, 1408-bit, and 1984-bit.
- Physical security considerations associated with guarding against leak of the private key or unauthorized access during storage, transmission, or usage.
- The quality of the two prime numbers used to create the public key modulus.

Potential risks associated with private keys include:

- Physical leakage of the private key.
- Actual factoring of the public key modulus.

- Bypass attacks on the IC card internal memory banks containing the private key.
- Defeat of the RSA algorithm; it is not currently computationally possible to defeat the RSA algorithm if the public key modulus is sufficiently large.

5.3.2.2 Certificates and Certification Authorities

In traditional cryptographic systems, key management primarily focuses on the creation and maintenance of secret key sharing practices. As the usage of public key and asymmetrical cryptographic technology continues to expand there will be changes in the content of how keys are to be managed. Within a public key cryptographic system, keys are produced in a pair at a time and are thus referred to as a key pair; the public key of a key pair can be released publically while the private key must be kept absolutely confidential; only the user of a given private key should know it.

Public keys must be transmitted to a using entity through credible channels. Although public key data does not need to remain secret or be encrypted, the entity that receives the public key must be certain that it has not been altered in anyway and has come from either a legitimate public key carrier or a credible authority. This is because a modified public key will not correspond to the public key known to the carrier of a non-modified private key. UICS specification debit/credit applications therefore solve this issue by utilizing a public key certificate, a common tool in public key cryptographic systems.

To understand certificates, we require a specific kind of public key cryptographic system referred to as a digital signature scheme. Using a digital signature scheme, an entity uses a private key to apply a signature to a message, creating a bit string referred to as a digital signature. This bit string—or digital signature—is actually a signed message and the output of a function of the entity's private key. Upon receiving a message and digital signature, the receiver can use the public key sent by the sender to verify the authenticity of that digital signature. It is in this way that a digital signature can be used to inspect the integrity of the source of a message and the data it contains. At this time this guards against a signer from denying the authenticity of a message bearing that signer's signature.

A certificate is a form of digital signature and can be used to verify the authenticity of a data source and the integrity of signed public key data. A given certificate includes a public key, links to associated data (which includes user name or personal identifier and certificate expiration date) and is signed using the private key of a credible certification authority (CA). This CA will issue a

public key to a user group in a credible manner; should any given user receive a credible copy of the CA public key then it will be capable of authenticating any certificate issued by that CA. This allows any user to receive a credible copy of the public key from any other user.

The IC Card Root CA of the UICS Debit/Credit Application Environment and the Root CA of Financial IC Card Debit/Credit Applications both act as certification authorities. IC Card Root CA utilizes the public key signatures of each issuer to create individual public key certifications for each issuer. Issuers also act certification authorities and utilize the public key and associated data signatures of each IC card to generate public key signatures. IC Card Root CA uses credible channels to transmit its own public key through acquirers to terminals and verifies the authenticity of issuer certificates; this way terminals receive credible copies of issuer public keys, which are then used to verify the authenticity of IC card public keys.

5.4 UICS Credit Application Verification Methods

5.4.1 Card and Data Authentication Methods

UICS debit/credit applications support the following methods for verifying the authenticity of cards and confirming that associated internal card data has not been modified since it was personalized by the issuer.

5.4.1.1 Offline Data Authentication

Offline data authentication for UICS debit/credit applications utilize static data authentication (SDA), dynamic data authentication (DDA), and composite dynamic data verification, also referred to as dynamic data authentication/application ciphertext message generation (CDA). Offline authentication refers to the authentication of card data by terminals and not issuers.

- For SDA authentication processes, terminals authenticate the static signature of card data as sent by a given card (that is to say, bearing the same signature for all transactions); this also verifies that card data has not been modified post-personalization.
- For DDA authentication processes, terminals authentic the dynamic signature for a given card, though this signature will be different for each transaction. This dynamic signature is created by the card using its private key, card data, and dynamic transaction data. DDA can authenticate verify

that the card is not an unlawful or fake card and that the data within the card has not been altered post-personalization.

- For CDA authentication processes, cards use their private keys to create a dynamic signature for a transaction—including for online encrypted messages—so as to provide DDA data protection. At the same time this guarantees that no intermediary machine between a card and terminal can change key transaction data.

5.4.1.2 Online Data Authentication

When online, UICS online data authentication is used so that the issuer can authenticate a given card and a card can authenticate a given issuer. At the same time, this also protects the integrity of data.

- During the online card authentication process, the issuer's online system confirms that the ARQC cyphertext message sent online by the card was created by using an accurate 3DES key for all important transaction data; this proves that the card is not a fake and that no transaction data has been modified.
- During the online issuer authentication process, the card confirms that the ARPC cyphertext message sent by the issuer was created using an accurate 3DES key and the issuer's secret key and confirms that the authorized response was generated by a lawful issuer. This also verifies that no data has been changed. Internal card management operations are conducted in accordance with message content, e.g. resetting PIN counters, etc.
- During the secure message process issuers use their secret keys to generate and transmit to cards an updated script protected by a MAC cyphertext message. Once the card has verified this MAC cyphertext message it will instruct the card to utilize the updated script; a successful verification of this MAC by the card signifies that the cyphertext message was generated by a lawful issuer and that the data transmitted during the process was not altered in any way. Security messages also are used to encrypt sensitive data like PIN updates between cards and issuers.
- For approved transactions terminals will transmit a cyphertext message TC generated by the card that includes settlement information to the issuer for verification; this serves as certificate verifying that the transaction has been completed.

5.4.2 Static Data Authentication (SDA)

SDA is a static signature verification mechanism based on public key cryptographic technology; terminals authenticate the personalized data of cards were created using the issuer's private key signatures to verify that the card's key data was not alternated after the completion of the personalization process.

Figure 1 illustrates the relationship between data and keys during static data authentication.

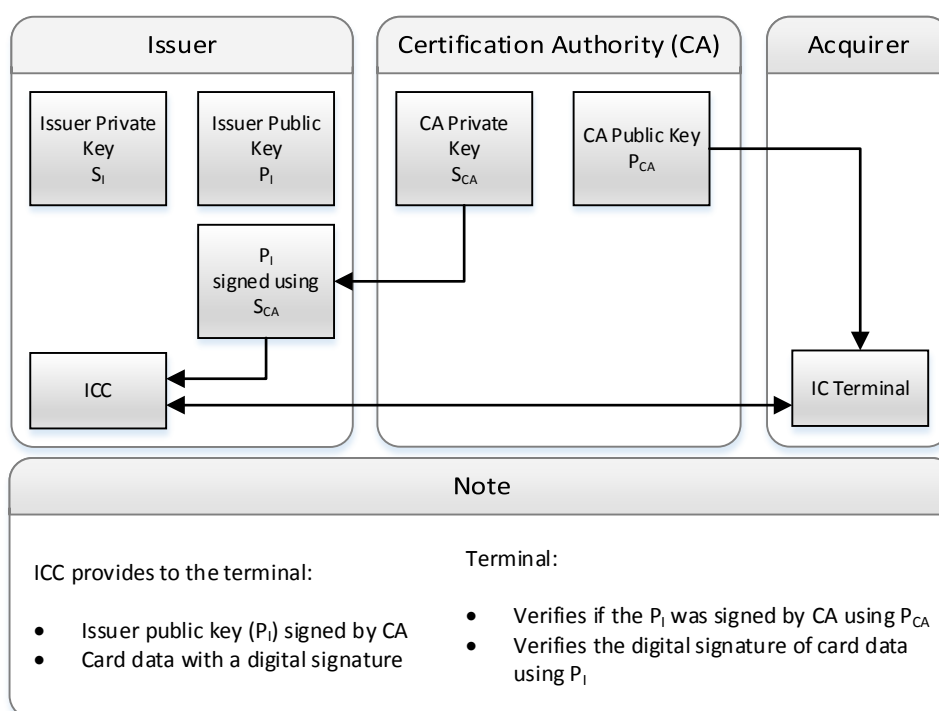


Figure 1 Static Data Authentication

In regards to offline data, as SDA is the verification of a card's static data signature it is possible that the lawful static data signature from a given card's internal memory could be copied to another and thus generate an illegal card copy. This type of illegal mechanism may lead to the offline verification of static data and allow for offline transactions. However, as it is impossible to copy the secret key (i.e. the 3DES key) from one card's internal memory to another, this type of illegal mechanism cannot be used for online transactions.

5.4.3 Dynamic Data Authentication (DDA/CDA)

DDA and CDA are based on the dynamic signature verification mechanism of public key cryptography. Terminals utilize the private key of a given card to verify dynamic transaction data and essential card data or dynamic transaction

data, essential card data and transaction cyphertext message signatures to determine the veracity of essential card data; as dynamic signatures require the signature of a card's private key and it is typically impossible to read this private key off of a card's internal memory, the usage of dynamic data authorization rules out the possibility of using a copied card to carry out offline transactions.

See Figure 2 for the relationship between dynamic data authorization data and keys.

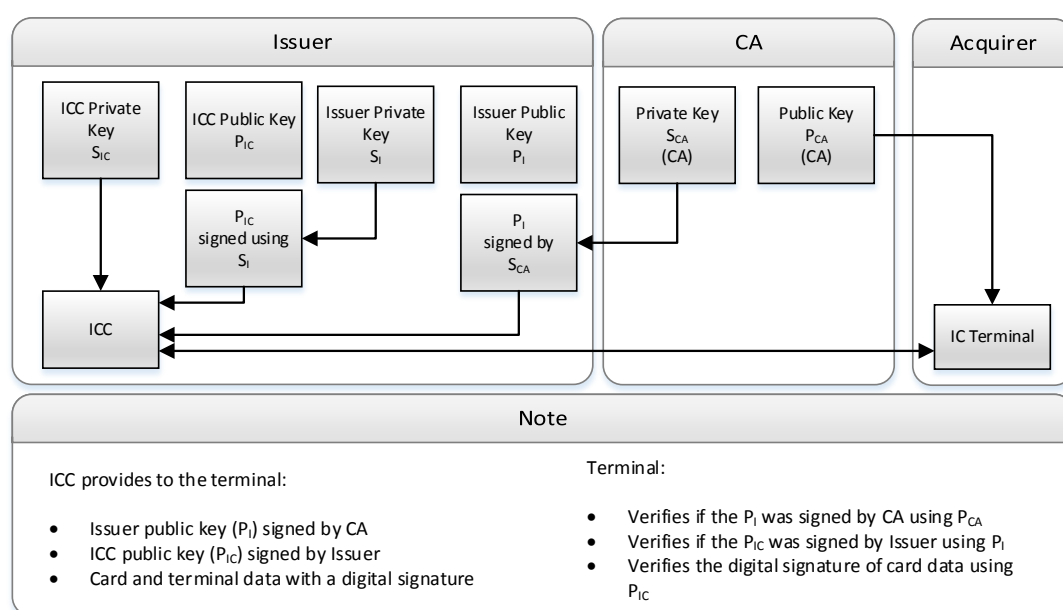


Figure 2 Dynamic Data Authentication

Aside from using cyphertext messages and other transaction data (such as transaction approval status) and dynamic application data signatures, CDA and DDA functionality is extremely similar. A unique characteristic of CDA is the ability to have terminals verify that the above data was generated by an appropriate card and that the transaction data was not modified as it was transmitted between card and the terminal.

5.4.4 Comparison of Authentication Methods

Table 1 below explains the security and protection supplied to card transactions by different authentication methods.

Table 1: Card Verification Comparison 1

	SDA	DDA	CDA	Online Card Authorization
Checks for illegal card static data	✓	✓	✓	✓
Checks for modifications made to card static data	✓	✓	✓	✓
Checks for illegal card copies		✓	✓	✓
Checks for outside attacks during card-to-terminal transmission processes			✓	✓
Can be used for offline transactions	✓	✓	✓	

It should be pointed out that SDA, DDA and CDA—three kinds of offline authentication methods—provide steadily increasing levels of transaction security. Table 2 below illustrates how these different methods affect cards, terminals, computer processing, and transaction times.

Table 2: Card Verification Comparison 2

	SDA	DDA	CDA	Online Card Authentication
Card		Must support RSA	Must support RSA	Must support 3DES
Terminal	Must support RSA	Must support RSA	Must support RSA	Must have online capability

	SDA	DDA	CDA	Online Card Authentication
Card Issuance and Main Computer System		Must personalize IC card public/private key pair	Must personalize IC card public/private key pair	Must personalize IC card secret key and support IC card transaction authorization
Transaction Time	Double terminal operation of RSA	Triple terminal operation of RSA and one IC card operation of RSA	Triple terminal operation of RSA and one IC card operation of RSA	Must receive online authorization

5.4.5 Cardholder Authentication Methods

UICS specification debit/credit applications support the following cardholder authentication methods:

- Handwritten signatures
- Online PIN encryption; cardholder enters online PIN into terminal; terminal PAD encrypts this online PIN and online transaction request message and transmits it to the issuer; issuer then verifies both cardholder and transaction request.
- Offline plaintext PIN; cardholder enters offline PIN into terminal, whereupon the terminal then transmits this PIN to the card; the card compares this offline PIN to the offline PIN stored aboard its internal memory. The card then informs the terminal whether or not offline PIN verification was successful.
- Offline plaintext PIN verification and addition of signature
- Signature
- No CVM required (CVM is considered to have passed)
- CVM processing failed (CVM is considered to have failed)
- Certificate presentation

5.5 Transaction Authorization System

Transaction authorization uses the following process: the authorization or rejection of a transaction by an issuer or issuer representative is completed by providing a response to an online transaction and authorization request sent by a merchant terminal through an acquirer to the issuer.

Online transaction authorization requests are composed of the ARQC authorization request message generated by a card; an issuer then verifies this message to confirm that it came from an actual card and that no transaction data has been modified. This online transaction request also includes the card and terminal identifiers produced by their respective offline processing results.

As a card is replying to the ARQC, the issuer can choose to generate an ARPC authorization response cyphertext message and transmit it to the card. The card verifies that this ARPC to determine that the cyphertext message has come from a real issuer and that the message data was not modified during transit.

Aside from the ARPC, issuers can also use issuer script commands to execute post-issuance updates to the card. For instance, the issuer can change the offline PIN or update a card's risk control parameters. The issuer can use the generation of a MAC cyphertext message to protect the integrity of the script command data; as the card executes the script commands sent by the issuer it will verify this MAC cyphertext message. Encryption is used during this process so as to protect the security of secret data.

6 UICS Card Issuer Security

This chapter contains information on what needs to be completed by issuers during the card issuance process and related security functions:

- Generate, manage, and securely store issuer public/private key pairs.
- Send issuer public key to IC card Root CA so as to receive its signature on the issuer public key certificate.
- Receive and store the issuer public key certificate and related IC Card Root CA public keys so as to verify issuer certificate signatures.
- Generate IC card public/private key pairs for DDA or CDA applications.
- Utilize issuer private keys to issuer IC card public key certificates or any corresponding SDA application-signed card static data.
- Generate and safely store issuer symmetrical main keys (secret keys).
- Use issuer main keys to export IC card main keys (secret keys) for online authorization and secure message applications.
- Send all keys required for card personalization processes to the issuer's card personalization system in a secure manner; this includes the input and output of issuer main keys (only when the issuer entrusts card online verification services to a third party).

6.1 Issuer Tasks

Issuers must complete the following tasks during both the card issuance process and throughout the card's effective lifetime:

- Pre-IC card issuance preparation tasks,
- IC card procedures, including card generation (SDA card) and issuer-supported static data verification,
- IC card procedures, including card generation (DDA and SDA card) and issuer-supported dynamic data verification,
- Tasks required for the issuer to supply the cardholder with a newly generated UnionPay IC card,

- Online transaction processing and card-support sequences, including verification of UnionPay IC card online cyphertext messages and cardholder online PIN, and the update of card application issuer script commands.
- Transaction settlements, UICS debit/credit application-supported transaction settlements, and clearing sequences.
- Invalidation; while completing card issuance process tasks and after the completion of issuer processes all keys will be invalidated and destroyed.

6.1.1 Card Issuance Preparation

Before card issuance can begin the issuer must first complete the following tasks. These tasks must also be completed when updating keys or a certificate has expired.

6.1.1.1 Asymmetrical (RSA) Keys

Key pair generation: issuers must securely generate public/private key pairs and must store and protect them in physically-secure memories. These public/private key pairs must be generated within secure encryption hardware using random or pseudo-random generators and standard primality testing procedures.

The public/private key pairs that must be generated by the issuer include:

- Issuer public/private key pairs: private keys are used to issue card static data and IC card public key certificates while public keys are transmitted to IC Card Root CA in exchange for a signed public key certificate.
- IC Card public/private key pairs: Issuers must write the IC card private key into the card during the card personalization process for DDA and CDA authentication. The issuer uses its private key to sign the IC card public key certificate and then write this certificate into the card during the card personalization process.

RSA public key modulus length (represented by bit value, such as 1024-bit, 1152-bit, 1408-bit, and 1984-bit) is determined by the card's security requirement plan for the entirety of the card's usage lifetime. It is strongly recommended that the issuer select a public key modulus long enough to resist RSA computational attacks, i.e. factoring of the public key modulus. In consideration of payment system efficiency, it is recommended that for the lifetime of a given issued card that IC Card Root CA and issuers use public key modulus lengths and corresponding private keys for issuer certificate signatures

that are in accordance with the security requirements of each specific card and not just the longest public key modulus length that is available. So as to increase total system efficiency and defend against risk, the length of the issuer public key modulus should be the same as the public key modulus used by IC Card Root CA to issue the issuer certificate.

Issuers should periodically inspect and evaluate the RSA public key modulus lengths that are in current usage; should it be found that a public key modulus length is no longer appropriate for that key's security requirements then the usage of that key should be stopped and an issuer public/private key pair with a larger public key modulus be introduced. In the same way, assessment should be made to determine if issuer public key length corresponds to the security of issued cards.

RSA key generation using primes should be generated using a prime randomly selected from at least 2_{100} prime numbers. There should be a difference of at least 2_{128} between p and q , which represent the two primes used to create the public key modulus.

The generation of primes used in the generation of public/private key pairs should be based on standard prime number generation algorithms; ISO/IEC 18032 may be consulted for more information regarding the generation of prime numbers. Operational efficiency is the most important element to consider when selecting public key indicators. Generally speaking, terminal RSA calculation time for choosing a public key indicator of $2_{16}+1$ is longer than a public key indicator of 3. However according to cryptographic theory, assuming a sufficiently large public key modulus, an IC card performing a RSA selection of a public key indicator of 3 will be just as effectively secure as a selection of $2_{16}+1$. For this reason, 3 should be chose as the public key indicator.

The random or pseudo-random process used for key generation should satisfy the following requirement: it must either generate an unpredictable key or must not generate a key that is more likely to be generated than other keys. Random or pseudo-random number generators should conform to both the ISO/IEC 18031 and the randomness testing requirements of the State Bureau of Commercial Cryptography.

The physical security device (i.e. the hardware security encryption module) used to generate and protect the integrity of RSA keys should be tamper resistant and conform with the security requirements of Section 8.1 – Secure Cryptographic Device.

It is recommended that issuers use different public keys to generate IC card data for different BIN numbers so as to limit the number of cards generated by issuers that use the same public key; this will reduce the number of cards that would be affected in the event that a public key was leaked.

Issuers should verify that issuer public key data has not been modified when transmitting issuer public keys to Root CA for the creation of issuer certificates.

Regarding payment system IC Card Root CA: different issuers should have different public/private key pairs.

For a guide on the key generation process, see Section 7.1.

Accepting IC Card Root CA public keys: issuers should accept and securely store one or more Root CA public keys and be in accordance with both UICS Specifications and Financial IC Card Debit/Credit Application Root CA Public Key Authentication Specifications when accepting Root CA public keys as well as verify the integrity and source of public key data.

Applying for and accepting issuer public key certificates. Issuers must send each issuer public key to Root CA and accept each returned issuer public key certificate.

When sending issuer keys to Root CA, issuers must ensure that Root CA is capable of verifying the completeness and source of issuer public key data.

After receiving issuer certificates issued by Root CA, issuers should use corresponding Root CA public keys to verify the authenticity of these issuer certificates. For technical and industry specifics, please see Financial IC Card Debit/Credit Application Root CA Public Key Authentication Specifications.

Issuer private keys and storage of issuer certificates: Issuers must store their private keys. These keys must be stored in an encrypted manner or stored in a secure cryptographic device; see 7.2.1 - Issuers Must Store Their Issuer Certificates.

6.1.1.2 Symmetrical (3DES) Keys

Generation of issuer main keys: Issuers must securely generate and store one or more issuer main keys for exporting the unique IC card main key for each card application. This requires that security hardware possess secure memory and utilize a random or pseudo-random number generator.

The 3DES keys used by UICS debit/credit applications are used for special transaction functions. Card main keys are distributed from issuer main keys and exported into cards during the personalization process. The main key for each card is unique.

Issuer main keys include:

- Issuer Application Ciphertext Main Keys: used for exporting card keys which are used to generate MAC during transactions, namely the Application Ciphertext message AC.
- Issuer main keys are used to protect the integrity (IMK_{SMI}) of secure messages, which are used to export card keys. These card keys are used to protect the integrity of the secure messages sent between cards and the authorization system during post-card issuance process. Namely card blocking/unblocking, updating special card data, and PIN changes.
- Issuer main keys are used to protect the secrecy (IMK_{SMC}) of secure messages, which are used to export card keys. These card keys are used to protect the secrecy of the secure messages sent between cards and the authorization system during post-card issuance process. Namely card blocking/unblocking, updating special card data, and PIN changes.

3DES keys should be generated in a physically secure device equipped with anti-tamper protection mechanisms; the entirety of the key should be generated using a segment compounding process using the key segments possessed by authorized individuals. The key compounding process should occur within the physically-secure device. Additionally, it should be guaranteed that knowledge of any subset of any key segment does not allow for the derivation of the entire key. While a key is being generated in segment form during the card personalization process, at least one key segment should be generated by an issuer employee.

The random or pseudo-random process used to generate keys should be such that either the prediction of any key is impossible or any given key be more likely to be generated than another. For more information regarding random or pseudo-random number generators, please see ISO/IEC 18031. Random or pseudo-random number generators should conform to both the ISO/IEC 18031 and the randomness testing requirements of the State Bureau of Commercial Cryptography.

It is recommended that issuers use different issuer keys for different BIN numbers so as to limit the number of cards generated using a given issuer key.

Keys should only be used for set encryption purposes and only for specified applications. That is to say, they should be used for application cyphertext messages, secure message integrity, secure message encryption, etc. Different encryption applications should use different main key exports.

Issuer symmetrical main keys should be updated annually. This does not mean that issuers must update the keys inside all issued cards, but that the issuer must be capable of managing different key forms. Should a given card and corresponding card key created by the export of this kind of key form become expired and not subject to dispute, then the issuer should destroy this issuer key form.

For more information regarding key generation, see Section 7.1.

- **Transmission of issuer main keys:** Should an issuer take the task of card personalization, card generation, or verification of online cyphertext messages and entrust it to either a third party or a separate system within its own processing centre, then that issuer must securely transmit the issuer main key to that third party or processing centre for so that IC card key exporting tasks can be accomplished. (See Section 7.2).
- **Back-up of issuer main key:** Issuers must back-up their issuer main key.

When backing up primary issuer main keys, it is recommended that a key encryption form with the same length be used, or alternatively, that the issuer main key be split into two-or-more person secure control mechanism where each entity control some segment of the key segment value. This process should be capable of being audited.

6.1.2 Secure Counting Devices

If secure counting devices are used to limit key usage, then the threshold value of the counting machine must be set before it is to be used.

This guide recommends the following classical offline secure counting machine threshold values:

- Offline PIN Decryption Limit = 1024
- ATC Limit = 20000
- AC and SMI Secure Counting Device Threshold Value Limit = 1024.

Note that different payment systems can support different threshold values.

6.1.3 Card Production (SDA)

Issuers should implement the following security-related steps for every SDA card that is issued:

- **Static Data Preparation:** The issuer will generate card data necessary for card personalization.
- **Static Data Signing:** The issuer will utilize an issuer private key to sign all of the card static data selected for that card's signature static application data.

The issuer should abide by UICS specifications and requirements when generating card static data:

- **Derivation of IC Card Main Keys:** The main keys used by IC cards for cyphertext messages and script message security should be the main keys of an appropriate issuer and should be exported using PAN, PAN sequence numbers, and other card data.
- **PIN Generation:** If a given IC card supports offline PINs then an offline PIN value should also be generated. The offline PIN should be identical to the online PIN.
- **Supplying Card Data for the Card Personalization Process:** Public key certificates, Root CA public key indexes, signed card static application data, dispersedly exported card main keys, exported key indexes (if used) and PIN numbers should all be transmitted to the card personalization system in a secure manner.

The issuer should securely transmit all data necessary for card personalization to the card personalization system and securely write it into the card.

Throughout this process, either MAC or some form of data signature should be used so as to guarantee that the data transmission process is secure.

The transmission of secret keys and PIN data must be completed in a secure, confidential manner.

6.1.4 Card Production (DDA and CDA)

Issuers should implement the following security-related steps for any card produced that supports dynamic data authentication:

- **Card Static Data Preparation:** The issuer will generate all static data necessary for card personalization.
- **Signed Static Data:** If SDA functionality is required, the issuer will use an issuer private key to sign selected card static data; this allows for the generation of signed static application data.
- **Generation of IC Card Public/Private Key Pairs:** The issuer should securely generate a public/private key pair that is unique to each card.
- **Signing IC Card Public Keys for IC Card Public Key Certificate Generation:** The issuer should use its primary private key to issue IC card public key certificates and IC card static application data. The issuer private key public key certificate corresponding to the issuer private key that is used to issue the IC card public key certificate should be written into the card during the personalization process.
- **Derivation of IC Card Main Keys:** Issuers should use an appropriate issuer main key to dispersedly export the main keys that a given card uses for ciphertext messages and script message security; card data with PAN and PAN sequence numbers should be used during the derivation process.
- **PIN Generation:** If offline PINs are supported then an IC card offline PIN should also be generated. The offline PIN should be identical to the online PIN.
- **Supplying Card Data for the Card Personalization Process:** Issuers should prepare and transmit secret keys and PINs exported by IC Card Root CA public key indexes, signed card static data, and exported key indexes (if used) to the card personalization processing system.

All of the card data necessary for card personalization should be securely transmitted to the card personalization institution and securely written into the card. All of the data required for card personalization should be protected either by MAC or a form of digital signature throughout the entirety of the data generation process, from the moment it leaves the data generation system until it is written into the card. The card secret key, card private key, and card PIN should all be protected by secure methods that protect data confidentiality.

After Card personalization, any record regarding IC Card private key should be destroyed by IC Card Issuer and Personalization Institution.

6.1.5 Card Issuance

Issuers should implement the following security-related steps after each card issuance process has been completed.

- **Preparation and Transfer of IC Card and PIN to Cardholder:** Once personalization has been completed, the IC card and PIN should be securely transferred to the cardholder; this transfer should abide by ISO 9564-1 specifications.

6.2 Post-Issuance Conduct of Issuer

The issuer should abide by the following procedures for each online transaction.

- **Encrypted Message Exchange:** As a part of the security procedures for (online-capable) merchant terminals transactions, both IC cards and merchant terminals may request the online authentication of cards.

Online authentication includes the transfer of application cyphertext messages (ARQC) from the card to the issuer. Cards utilize IC card main keys (dispersedly exported from issuer main keys during the personalization process), work key encryption cards dispersedly exported from IC card main keys, terminal data, and transaction data to generate these cyphertext messages. Issuers should use the same keys to verify this ARQC.

Issuers may use the same key used to verify ARQC to generate a response to the cyphertext message (ARPC). ARPC is an issuer-encrypted ARQC and the data result of an authorization response or card status update. Cards use the same keys used to verify ARPC to verify that this response is from a lawful issuer and that the data has not been modified during transmission.

- **Secure Messages:** A part of the card management process, secure messages (encrypted messages created using dispersedly exported IC card keys) can be used to transfer data updates from issuers to cards. These updates may include PIN updates, PIN counter resets, the blocking or unblocking of applications, card blocking, and the modification of a card's risk management data.

- Cardholder Verification: Within an online environment (such as an ATM), an issuer will be requested to verify the encrypted PIN that is sent following the transmission of an authorization message.

6.2.1 Transaction Clearing

The issuer must implement the following security-related steps for each approved transaction.

- TC Processing: The result of each IC card transaction, IC cards use card secret keys to generate a transaction cyphertext message (TC) and then transfer said message to the merchant terminal. This TC is then sent by the merchant terminal to the acquirer who then sends it back to the issuer as part of the transaction clearing process. As part of the transaction clearing and settlement process, the issuer is required to verify the accuracy of the TC.

6.2.2 Key Invalidation

This section provides guidance on the invalidation and destruction of keys and key-related materials.

6.2.2.1 Key Retention

This section specifies that before a key is determined to be no longer required that it should be kept or retained; once it has been determined that a key is no longer needed it should be either invalidated or destroyed.

Issuer asymmetrical key pairs should be retained using the following methods:

- Issuer private keys should be retained until they are no longer required for the personalization of any card; that is, until the expiration of the issuer public key certificate that the private key corresponds to.
- Issuer public keys should be retained until the card verification processes that utilized the key are no longer in dispute.

Issuer symmetrical key pairs should be retained using the following methods:

- Issuer main keys should be retained until it is no longer possible that any of the cards using card keys generated using the export of those card main keys can be disputed.

6.2.2.2 Key Destruction

The destruction of key-related materials should be completed using a process that appropriately destroys the medium used to store the material.

- If the key is stored inside an IC card, a hole can be made in any of the chips within the card, which causes the physical destruction of both the chip and any data contained within it. Expired cards not within issuer control may have not been effectively destroyed and retain some functionality, even if merchants do not accept the cards for processing transactions.
- For key materials contained in a magnetic medium, residual magnetic signatures should be removed by degaussing. Mobile mediums should be physically destroyed (such as by being cut into pieces). Note: modern hard drives possess reimage functionality which may reduce the effectiveness of the destruction process; physical destruction or data-removal technology should be utilized in these cases.
- SCD security mechanisms should be used for the destruction of discarded keys contained with an SCD.

An independent third party—such as an internal auditor—should monitor and record the key-related material destruction process. This record should then be retained for a period of time in accordance with the issuer's policies regarding document storage.

7 Key Management Practices

The secure implementation of the UICS Debit/Credit Application Transaction System depends on whether or not the issuer generates and manages the keys required for UICS debit/credit applications in accordance with the following general requirements.

For more information regarding the management of retail bank keys, see ISO 11568.

7.1 Key Production - General Guide

The following general guide should be used when generating symmetrical and asymmetrical key pairs. Many of the processes rely on random bit generation or prime generation.

The primary requirement of UICS Debit/Credit Application Card Issuance System Key Management is the secure generation of keys. Keys within the UICS Debit/Credit Application Transaction System are stored in the following locations:

- IC Card Root CA System
- Issuers
- Card Issuance System
- Cards
- Within terminals;

Keys may be used for card issuance processes or card transaction processes. These keys are generating using the following two methods:

- Singularly-generated keys: keys generated for the protection of the card production process
- High-volume card key generation:

Keys placed in storage to restore systems in the case of an accident; these keys typically must be stored by in segments and their operation must be conducted by two-or-more person control mechanisms. Key production processes must be recorded in daily journals for auditing purposes. High-volume card key

generation occurs within card production systems equipped with security measures; the entirety of the security process must also be recorded for auditing purposes.

All key production must be completed in accordance with a defined process; the security of this process must be defined and all associated system roles must be in place before production begins. Process specifics will depend on the types of application systems that are affected.

UICS Debit/Credit Application Main Keys should be generated in accordance with a pre-defined key production process and this process should occur in a secure card production environment. The encryption module used during the key production process should be in accordance with the relevant requirements of the People's Bank of China and the State Commercial Cryptographic Management Bureau. The pseudo-random number generator used for the key generation process should also be in accordance with the relevant requirements of the State Commercial Cryptographic Management Bureau.

The environment used for key generation must be secure and appropriate; the equipment and equipment configuration used must be capable of guarding against multiple kinds of attack, including electromagnetic radiation analysis, etc.

For more guidance regarding random bit generation, see ISO/IEC 18031; regarding prime generation, see ISO/IEC 18032; randomness testing and prime testing procedures should be in accordance with the testing specifications of the State Commercial Cryptographic Management Bureau.

All key generation should be completed in accordance with a predefined process. This process should be clearly defined and assure that all process hosts have an appropriate amount of time to get into proper positioning. The details of this process will depend on the specific system for which the keys are being generated. The security of the key generation system should be guaranteed. The security of the equipment, method, and composition of the system should be guaranteed such that the entirety of the key generation process is fully protected from electromagnetic attacks and other forms of special bypass attacks.

All materials used in the key generation process and their respective expiration dates should be clearly defined so as to guarantee the following:

- Generated keys are not used for anything other than a predefined purpose,

- Testing and key generation materials do not become confused with one another,
- Replacement of keys when necessary:

A specific plan for each key type should be created that defines when a key is created, how it is generated, where it is stored, how it is stored, and the various roles required for its generation. This process should be clearly defined and easily followed by all roles. Individuals (or "key managers") or control a key component (key segment) will not also have knowledge about the details of this process or always need to use it.

This process should clearly delineate who has the authorization to conduct its operation. It should define how the key generation system is established and how the system is purged of information relating to key data once key generation has been completed. How key segments are stored and who conducts this storage should be clearly defined.

This process should define how many key data copies can be made for a given generated key and how that key data is protected. Note: many keys require at least two copies of key data; one for the generation system and one for off-site storage. The backup of both copies require the same level of security.

The execution of this process should generate a record that allows an auditor to determine whether or not a key was generated, stored, and used correctly based on authorizations and participant signatures. The process should also guarantee the security of key data. The checksum value located in the hardware security encryption module and the MAC value generated outside of this encryption module are to be used for this purpose.

If the checksum value internal to the hardware encryption module is used, its calculation should be directed towards both key subsegments and the entirety of the actual key value. The checksum value calculated for key subsegments should be sent as one transmission.

If any secret key or private key should be found to be located outside of physically secure equipment or if it should be discovered or suspected that the all subsegments of a given key are controlled by a single person, then that key should be considered as having been compromised.

7.1.1 RSA Key Generation

This guide suggests the RSA public key algorithm key pair found in UICS Debit/Credit Applications be generated in a hardware security module and be randomly selected from amongst 2^{100} prime numbers. Additionally, for any public/private key pair, create the public key modulus M created by the primes p and q ($p \cdot q = M$) should have a different of at least 2^{128} . A prime generation algorithm approved by the People's Bank of China and the State Commercial Cryptographic Bureau should be used, such as using Fermat's primality testing and the Gordon algorithm to select strong primes and then testing those selected primes using the Miller-Rabin test. The number of iterations required for each test will depend on key length. For example, if the prime number error rate is required to be less than 2^{-100} , then for a 1024-bit RSA key the Miller-Rabin test should be iterated 7 times. For issuer public key pairs, even more iterations should be chosen because there is comparatively fewer generations of issuer public key pairs. A more deterministic method of prime number generation can be used as well, so long as the primality of the values created can be proven.

7.1.2 Certificate Generation

Certificate generation should comply with UICS and Financial IC Card Debit/Credit Application Root CA Authentication specifications. Additionally, if an encrypted hardware module is used to generate public key certificates then the certificate should be inspected before it outputted from the device so that fault attacks can be avoided.

The crux of public key certificate integrity is the issuer public key certificate issued by IC Card Root CA. An issuer public key certificate is created when an issuer send its public key to IC Card Root CA who then issues a certificate based on that public key. Aside from issuing issuer public key certificates, IC Card Root CA public key authentication services are also responsible for issuing Root CA public keys. Issuers must verify that any Root CA public keys received are authentic while Root CA must in turn verify the authenticity of the issuer public keys that are received before issuing a public key certificate. China UnionPay Financial IC Card Credit/Debit Application Root Ca Public Key Services provides tools and methods for accomplishing these objectives.

7.1.3 Symmetrical Key Generation and Dispersal

All of the symmetrical algorithm keys used for UICS credit/debit applications should be at least double-length, i.e. 128-bit (as compared to DES single-length 56-bit keys). All of the symmetrical algorithm keys associated with card transactions must meet this standard. All of the symmetrical algorithm keys used

for other card production processes (such as transmission or secure communication keys) must also be at least 128-bit. All keys used to protect the card production process should be generated securely.

The usage of the weak and half-strength keys of the DES algorithm should be avoided. There are 16 weak and half-strength keys in DES, thus making the probability of one being generated 2^{-52} . If it should be discovered that a random number generated has generated one of these keys, then the possibility that the generator has become ineffective is much larger than other probabilities and should therefore be inspected. For dispersion algorithm keys, due to the fact that one cannot avoid weak key generation through simple re-generation without changing dispersion parameters such as main account number PAN or PAN sequence numbers, it is not necessary to conduct weak key or half-weak key inspections of keys generated by dispersion algorithms.

The generation of UICS debit/credit application symmetrical keys should be exported and stored in a dispersed manner; only the combined action of all holders of all the segments of a given key or all of the of all key values for a pre-defined subset of a given key value will be capable of restoring that key. It should be guaranteed that the holder of secured key segment be incapable of accessing the key segment values of other key segment holders. It is recommended that symmetrical algorithm keys be divided into at least two segments.

7.1.4 Random Number Generators Used for Key Generation

This guide required that the random number generators used to generate UICS debit/credit application keys be in accordance with the requirements of the UnionPay International as well as Level 3 requirements of FIPS140-2 Standards. Key hardware modules should be in accordance with the relevant testing requirements of the UnionPay International as well as the Level 3 requirements of FIPS140-2 Standards.

7.2 Key Storage and Transmission

7.2.1 Key Storage

Keys used for UICS Debit/Credit Application Systems should be stored in the following locations:

- Issuer card generation personalization data systems
- Issuer card generation systems

- Card internal memory
- IC Card Root CA system and associated service systems
- Acquirer terminals
- Issuer main system encryptor
- Key transmission systems,

Require back-up storage of IC card keys. All key storage processes must be kept secure, complete, and usable.

7.2.1.1 Hardware and Software

Keys are usually stored inside hardware (such as within an encryptor) or within software (such as within a main computer system). In actuality keys are generally stored on magnetic mediums (such as magnetic disks) and can be transferred to variable silicon memory, or long-term, stable silicon memory. Keys should be protected by a variety of physical protection methods, such as:

- Encryptors or IC cards possessing anti-tamper devices
- Keys stored on magnetic mediums on encryptors, IC cards, or main computer system operational systems equipped with logic protection should only be stored for short periods as the lifetime of magnetic mediums is only a few years. Keys stored in silicon memory can be stored for longer periods of time, maintaining stability for at least a decade. However a key is stored, is must be provided with adequate leak protection. Storage devices should be outfitted with physical security methods (such as placing anti-tamper storage devices inside a safe, etc.) The integrity of a key should be inspected before being exported from the storage medium, especially encrypted keys stored inside databases. This is because encrypted keys can still be altered even if they cannot be directly read.

The data integrity of keys stored inside secured devices (such as IC cards) can usually be inspected using non-key based comparison and inspection techniques that check for unusual circumstances. However, data security should be authenticated using MAC message authentication cyphertexts when a key is in an alterable state (such as when it is located within a database).

7.2.1.2 Access control

Access to all keys stored outside a hardware-based secure encryption module should require at least a two-person control mechanism.

7.2.1.3 Hardware-Based Secure Encryption Modules and Integrated Circuit Secure Memories

Hardware-based secure encryption modules and integrated circuits are similar in many ways, but very different in others. Generally speaking, hardware-based secure encryption modules include a required number of independent memory and processing components and transmit keys between components using datalines. This means that hardware-based secure encryption module memory banks must be formatted should a data leak be discovered, thus making it important that their design consider protecting against electromagnetic leaks. Hardware-based secure encryption modules are generally designed to operate within a secure environment. IC cards on the other hand are designed to be used in unprotected environments and memories installed with anti-tamper devices.

7.2.2 RSA Key Storage and Transmission

Public keys should be capable of being transmitted in a way that allows for the authentication of the integrity of key data and the authenticity of the data source. For specific information see Financial IC Card Debit/Credit Application Root CA Public Key Authorization Specifications. Public keys should be located either within a signed public key certificate during transmission, under MAC protection using purpose-generated MAC keys, or under dual control (e.g. the independent transmission of checksum values and public key data). MAC algorithms should follow the methods directed by the received of ISO16609 public key's regarding to the authentication of received public key data, accepted Root CA public keys, shared MAC keys, and the integrity and source of checksum values. Private keys should be kept confidential; should private keys need to be transmitted then the integrity and confidentiality of that transmission must be guaranteed. Private Key transmission mechanisms include the following:

- Transmission of a private key while stored within secure encrypted hardware,
- Encryption of private key transmissions using a symmetrical encryption algorithm of at least the same level of strength as algorithms used to protect private keys,

- If a derived private key is to be transmitted then a dual control mechanism should be put in place that places each key segment under independent control.

Business recovery policy should require that a back-up be made of the public/private key pair and that this back-up be secure. Should catastrophic system failure occur, this back-up will be required to restore the system to normal operation. System key backups should also be placed under secured protection. Note: should issuer private keys be completely erased this will not affect the usage of already-issued cards; the issuer should generate a new issuer public/private key pair and then receive a new issuer certificate; the generation of new issuer public/private pairs will not affect card usage.

7.2.3 3DES Key Transmission and Storage

It is possible that 3DES keys will need to be transmitted and stored. For instance, issuers may transmit 3DES keys to a third party processing institution or card personalization firm. When 3DES keys are being stored or transmitted the following measures should be implemented so as to limit the risk of a leakage of key data.

Plaintext 3DES key data can be transmitted and stored while located within secure hardware or within an IC card with secure hardware protection.

If the 3DES key is not located in secure hardware then the following transmission and storage methods should be used:

- 3DES key data should be divided into two-or-more end segments utilizing dual control with key data segments under independent control, or;
- Located inside of a cyphertext message (e.g. an encrypted 3DES key); this message is generated using a pre-established system transmission keys or stored key.

7.3 Key Management Practices and Responsibilities

7.3.1 Key Management Personnel Planning

Personnel responsible for the management of key encryption, key segmentation, secure hardware and other key devices should be established by all participating parties; these parties include issuer, third-party processing institutions, and card personalization firms.

While determining what personnel are to be responsible for the management and control of key materials and secure hardware an appropriate amount of control systems should be introduced that guarantee that no unauthorized person have the ability to access either key materials or secure hardware.

Key management personnel should be trusted employees and not temporary contractor employees or external consultants.

So as to ensure business continuity back-up key management personnel should be planned in addition to the management personnel already named; the selection of these key management personnel should be based on the same basis as the original key management personnel.

7.3.2 Key Management Functions

Key management is extremely important and forms the basis of the issuer security agreement. The keys managed by key managers are the most important keys utilized by both issuers and card during usage and transaction processes.

Every issuer should abide by the following rules when auditing internal key management procedures and all associated key management personnel.

- The responsibilities of key management include control of key materials, verification of key materials, and ensuring the secure storage of key materials.
- Key management personnel are responsible for:
- Accepting and securely storing key segments and secure hardware and conducting verification and authorization of received key materials.
- Maintaining records and logs to sustain traceability of key material access and usage, including access times, dates, purposes, and secure return and storage of used key materials.
- Verification of all transmitted key materials sent by personnel outside the scope of issuer control.
- Witnessing the destruction of all expired or voided key segments.
- Placing key materials into secure encryption modules when necessary.
- Guiding and monitoring the destruction of voided key materials.

7.3.3 Process Used When Delivering Key Materials to Third-Party Institutions

Two duplicate forms should be created whenever delivering keys to a third-party; these two duplicates should be sent at the same time from the key sender to the key receiver. These duplicates should confirm the identity of the sender as well as the delivered key materials. The key sender should audit these duplicates before they are sent and affix a signature to each. Once the key receiver has received both the delivered key materials and the duplicate forms it should immediately compare the key material with the verification listed on the duplicate form, and then affix a signature to both duplicate copies and return them to the sender.

When delivering key segments or other key-related data to a third-party, each key segment should be sent to different people using different methods, such as express mail, registered mail, messengers, etc. During the delivery process, each key segment should be located within either secure hardware or an IC card; this will satisfy registered mail requirements.

The receiver of the key material should be informed by the sender about what key segments are going to be sent before the transfer is made.

7.3.4 Physical Key Security Protection Process of Third-Parties

Once key materials have been delivered, the person in charge of managing the key materials should immediately inspect the package and envelope that the key materials were contained in for signs of unauthorized access and verify the contents.

Should the receiver doubt the integrity of received key materials, then the sender of those materials should be notified immediately. With the cooperation of the receiver, the sender should determine the future state of this key material. Any decision or reasoning regarding whether or not this key material should be used must be recorded in written form and with copies kept by both receiver and sender.

Should the key material hardcopy need to be stored at a pre-determined time, then the hardcopy of all key segments, secure hardware, or IC cards should be stored within a physically-secure location inside a package or envelope affixed with an anti-tamper device. Separate key segments should be stored in separate physically-secure locations; these locations must not allow mutual access between them.

Serialized anti-tamper envelopes should be continuously protected within a physically-secure container, and access to these containers should be restricted only to either pre-determined key management personnel or backup key management personnel. Each and every access to these materials should generate a log entry that includes access time, data, envelope serial number, usage, and the signature of the accessing personnel. These logs should be made available to any appropriate institution that requests them for auditing purposes.

Key materials should be placed back into their tamper-resistant container and then returned to a physically-secure location as soon as the accessor has completed all necessary tasks.

8 Hardware and Software Security Considerations

8.1 Secure Cryptographic Devices (SCD)

Within ISO standards (such as ISO13491), a secure cryptographic device (SCD) refers to a common type of equipment that is used to supply a certain degree of protection for cryptographic calculations. High-end SCDs are a series of hardware modules that are connected to computer processors, personalization equipment, and authorized systems; low-end SCDs are individual secure hardware tokens and IC cards.

This section of the guide provides readers with a more complete understanding of the security attributes implemented by issuers for the required management of risk within transaction systems.

8.1.1 Anti-Tamper Requirements

Anti-tampering can be divided into two security-related areas: physical security and logical security.

8.1.1.1 Physical Security Attributes

Physical security includes the following attributes:

- Protects against penetrating attacks like the deletion of sensitive data, etc.
- Protects against the unauthorized leakage of sensitive information.
- Protects against electromagnetic, power overload, or other bypass attack attempts on the cryptographic algorithm.

Secure cyphertext devices should effectively protect against penetrating attacks, utilize enough physical protections such that penetrating attacks become ineffective, or actively detect penetrating attacks and deploy countermeasures. While operating as designed within a set environment, a successful penetration attack on a secure cyphertext device may cause keys or other types of sensitive information to be deleted.

To detect keys or other sensitive information or changes within a secure cyphertext device it must be placed within special-purpose analytical inspection equipment. Secure cyphertext devices should be equipped with effective anti-tamper measures that effectively guard against the insertion of conducting wires or the connection of other kinds of listening devices; the secure cyphertext

device should destroy the functionality of any module that detects this kind of intrusion. The detection of sensitive data from within a secure cyphertext device requires high-technical capabilities, highly professional equipment, and a high cost.

Secure cyphertext devices should utilize the following kinds of passive physical shielding:

- Shielding should be capable of blocking any x-rays or other types of electromagnetic radiation or externally-applied radiation which may be used to detect cryptographically operations taking place within the device.
- Privacy shielding should be capable of preventing key data from being accessed by or visible to work personnel during the manual loading of said keys.

The security of a secure cyphertext device is at least partially dependent on the security of its operating environment, so it should be guaranteed that unauthorized persons either be incapable of moving the device from its operating environment or that such removal trigger the device's anti-tampering devices.

8.1.1.2 Logical Security Attributes

Logical security includes the following attributes:

- Verification of software integrity.
- Design functionality should guarantee that the functions of any single device or combination of devices be capable of compromising confidential information.
- Mechanisms should be in place that guarantees that keys are only used for their designed purpose.
- Protections should be put in place that guard against disturbance-based or sequence execution-based bypass attacks on the device during operation of cryptographic algorithms.
- Operation of the device during sensitive states should require dual-operation control mechanisms.

- Authentication technology for the authentication of data and software updates.

8.1.2 Key Storage – General Guide

This section supplies general guidance for the storage of both symmetrical and asymmetrical keys.

8.1.2.1 Hardware Storage and Software Storage

Keys are generally stored either in hardware such as within secure modules or inside software such as within a main computer system. In actuality, keys are stored using the following mediums:

Keys are typically stored in magnetic mediums such as magnetic disks, though they can also be stored in silicon memories for brief storage and in long-term silicon memories such as IC card for long-term storage. During the key storage process, a series of physical measures are taken to protect the key. For instance, hardware security modules and IC cards are equipped with anti-tamper devices. The operation systems of hardware security modules, IC cards, and computer systems can also be equipped with logic protection measures.

Key storage in magnetic mediums should be considered short-term as the lifetime for most mobile magnetic mediums is no more than a few years. In comparison, storage in non-volatile silicon-based memories can have a storage lifetime of at least a decade. However keys are stored, the storage medium should be such that key security cannot be compromised. Storage devices should have adequate physical protections such as anti-tamper devices and a physically-secure operating environment.

The integrity of key data should be verified before being placed into storage.

Verification of key data integrity before being encrypted and placed within a database is an extremely important part of the storage process. Key encryption prevents unauthorized persons from accessing specific key values, though replacement of the encrypted key data may result in tampering attacks. For keys stored in hardware security modules such as IC cards, the usage of non-key based verification and inspection technology should be sufficient. MAC verification should be used to protect data integrity for keys stored in databases or other similar mediums which do not provide basic protection to keys.

8.1.2.2 Hardware Security Modules

Generally speaking, a hardware security module (HSM) contains separated storage and processing components; keys are transmitted between components using data wiring internal to the module. Therefore, inspecting for key leakage requires clearing the memories of the storage component. At the same time, the hardware design of hardware security devices should guard against electromagnetic probing and other forms of physical bypass attacks.

The circuit boards inserted into the PC used for the processing of keys should be considered a type of hardware security module and should be held to the same protection standards as other hardware security modules. The primary objective of cyphertext security modules is to protect any keys stored within them; if the computer system acting the hardware security module is itself unsecure then potential attackers will be capable of compromising keys by going around the hardware security module and attacking the software of the computer system.

It is very important that the key used to process the application cyphertext messages used in card transactions not be used for other purposes. That is to say, an authorized individual should be not able to conduct a large amount of unauthorized cyphertext operations (for instance, the unsuccessful verification of a MAC generated by a single lawful card or the inappropriate generation of a large amount of MAC-protecting secure messages or ARPC from a single lawful card).

Issuers should utilize API within hardware security modules to export card generation keys and generate issuer scripts. Because data components can take different forms, in some circumstances the API used within a system to connect to a hardware security module supports a flexible range of offset for inputted data; this ability may be utilized by attackers by using software that interacts with the API in such a way that collects and compares API responses. If an encrypted key is used for both key transmission and key management then this type of attacks may be possible. In these circumstances, either a special type of key management mechanism should be used or a key should be designated for a designated purpose and not used for anything else.

Key materials should be forbidden from being generated on normal personal computers or other types of non-secure devices.

Effective access control measures, software management controls, log records, and continuous monitoring should be implemented for all hardware security modules. The API of hardware security modules within a given system should only support real, accepted data forms and blocks everything else.

8.2 IC Card Application Security

Issuers are responsible for selecting an appropriate chip card for cardholders. To protect the interests of People's Bank of China, UnionPay, and Everbright Bank, PBOC provides an auditing process for assessing secure products. All relevant applications products must receive PBOC audit approval.

Different from hardware security modules that are designed to be used in secure environments, IC cards are designed to be used in unprotected environments. Generally speaking, the security of IC cards depends on onboard anti-tamper devices for the protection of sensitive information.

The management of card application security includes IC card platforms, application development security, and post-issuance card application data security. Although specific details are outside the scope of this guide, PBOC application development must abide by the following rules. It is very important that rules are followed and are subject to confirmation during IC card security audit assessment processes.

- Card application development should occur in a strictly managed and audited environment so as to dispel the possibility of unwanted application characteristics such as malicious coding and redundant test functions.
- Cyphertext functionality should be designed to reduce the probability of key leakage by time and power analysis methods by the greatest extent possible.
- The usage of individual keys should be effectively limited by process key processing, security counters, and other related mechanisms.
- Issuer card application functions should be limited so as to eliminate the possibility of unauthorized card data operations; card application data should be subject to integrity assessments on a regular basis.
- Issuers should only use IC card platforms that are approved by the People's Bank of China.
- Card application software should only be developed in a strictly controlled environment. This environment should be not only physically secure but utilize effective processes that guarantee the integrity of both application data and application source cyphertexts.
- Data used by card for personalization purposes should be managed in accordance with the data and IT security policies of the issuer. Confidential

application data—including keys, PIN numbers, and other types of secret data—should remain encrypted at all times and possess effective data integrity protections such that no participant be capable of direct access to confidential data plaintext.

- Confidential application data should only be used within the scope defined by UICS Debit/Credit Application Processes and should never appear outside of it. In the same sense, updating, resetting, or incremental data processing of any data not defined by UICS Debit/Credit Applications is strictly forbidden.

8.3 Fraud Detection

Issuers should process authorization requests using card data contained within a given card's online transaction request message in accordance with their own risk assessment procedures. Card offline processing results also appear in the terminal verification result (VTR) and issuer application data (IAD) contained within offline transaction authorization request messages. When making an authorization decision, issuers should consider all possible types of negative results, including offline PIN verification failures, etc. Other authorization domains (such as ATC, etc.) should also be assessed during the authorization decision process. Issuers should also verify the personalized data sent by the card (such as AIP, etc.) and confirm their authenticity.

Issuers should actively monitor PBOC debit/credit application transactions so as to detect any fraudulent usage IC cards, terminals, or fake cards. The scope of monitoring conducted by the issuer should include the following:

- **Offline Indicators Sent Online**—when a policy decides to approve or deny a transaction, IAD or other indicators sent through online data should be sent using online systems. These indicators may indicate offline authentication failure, offline PIN verification failure, bypassed offline PIN verification, or other offline processing results. These results may indicate fraudulent activity.
- **Offline Cyphertext Message**—Verify the authenticity of authorization requests and clearing messages so as to confirm a card's legal status.
- **ATC**—Inspect copied ATC and ATC values that are either smaller than the ATC of the previous transaction or when the interval between the current ATC value and the ATC value of the last transaction is very large. Note: When an authorization request is being repeated after the previous PIN authorization failed and a successive online connection has been made, the

copied ATC used in the authorization request for the new PIN verification will be lawful.

- Using a Terminal to Read Chip Card Magnetic Strip Data—although this situation may indicate fraud, it is possible that it has been caused by a read error of the chip card data.
- Inconsistent Chip Card Data—Verify whether or not the chip card data received during authorization requests and settlement messages is the same as both the card's personalized data and updated data.
- Script Commands—issuers should guarantee that card transaction messages sent by the issuer are not read by cards or applications that have already been blocked by issuer script commands.

Execution of the following operations is necessary to avoid fraud and credit damage:

- Following card theft, the card and all associated card applications should be blocked.
- A lawful cardholder's lowest limit amount should be adjusted to reflect their current offline transaction capabilities.

9 UICS Debit/Credit Application Keys

This section gives the keys used by UICS debit/credit applications

9.1 UICS Specification Debit/Credit Application Keys

9.1.1 3DES Main Keys

Double-length 3DES derived cryptographic algorithm and keys that are used for protecting the online transaction messages of UICS debit/credit applications. Issuers should use secure methods to generate and transmit 3DES main keys. Issuers must generate and utilize a certain number of issuer 3DES main keys and use card PAN/PAN sequence numbers to export the unique 3DES main key of a given card. Card main keys are securely written into a card during the card personalization process. Card personalization systems and transaction/authorization systems require the installation of issuer main keys.

The 3DES keys used during card transaction processes are the exported results of key segmentation algorithms conducted in accordance with UICS specification definitions using an IC card's main key. IC card main keys are exported issuer main keys created during the card personalization process. Process keys—only created when needed—are dispersedly generated from main keys. Issuers are not required to store IC card main keys because IC card main keys can be exported using issuer main keys.

The issuer main keys used by UICS debit/credit applications include:

- Issuer Application Cryptogram Main Key IMK_{AC}

An application cryptogram (AC) such as ARQC, ARPC, AAC, or TC; this is a Message Authentication Cyphertext (MAC) calculated using process keys. These process keys are dispersedly exported by a corresponding IC card main key MK_{AC} , while this IC card main key MK_{AC} is in turn exported from IMK_{AC} and written into a card during the personalization process. AC is used to verify the authenticity of transaction messages sent between card and issuer transaction authorization systems.

- Issuer Secure Message Main Keys IMK_{SMC} and IMK_{SMI}

Issuer secure messages are used to securely transmit messages during special post-card issuance transactions between the card and transaction authorization system (such as card locking and unlocking, card data upgrades, PIN changes, and PIN unlocks). Different keys are used for message encryption and

maintaining message integrity during this process, though both are dispersedly exported using IC card main keys. Issuer secure message main key IMK_{SMC} is used to export card secure message encryption main key MK_{SMC} , while issuer secure message main key IMK_{SMI} is used to export card secure message integrity main key MK_{SMI} . Secure message encryption uses secure message encryption process key SK_{SMC} ; this process key is exported from MK_{SMC} . Secure message MAC calculations use secure message MAC process key SK_{SMI} ; this process key is exported from MK_{SMI} . Secure messages are used for special issuer processes between cards and issuer transaction authorization system. This includes:

- Card locking
- Application locking and unlocking
- Card data updates
- PIN changes or unlocking

9.1.2 Card Static Data Authentication (SDA) Keys

SDA requires the issuer to utilize the private key of the issuer's RSA public/private key pair to sign special static data and write it into the card during the card personalization process. First, the issuer generates an issuer RSA public/private key pair and then uses the public key to issue an IC card Root CA public key certificate while the private key is used to sign the static data. The issuer then takes the issuer public key certificate and signed card static data and writes it into the card while the card is undergoing personalization. For more information on issuer certificate and signed static data formats, see UICS Security Specifications and Financial IC Card Debit/Credit Application Root CA Public Key Authentication Specifications.

Terminals will execute the following steps during the card SDA verification process:

1. Issuer public key certificates are read from the card.
2. The issuer certificate is verified using the IC Card Root CA public key certificate stored within the terminal.
3. The issuer public key is resolved from this issuer certificate.

4. The static data signature is read from the card and verified using the issuer public key. See Financial IC Card Debit/Credit Application Root CA Public Key Authorization Specifications for more information.

9.1.3 Card Dynamic Data Authentication (DDA) Keys

DDA requires that each card have its own RSA public/private key pair and that the public key possesses a public key certificate issued by an issuer in accordance with UICS security specifications. The card private key is to be stored aboard the card itself; during DDA processing this private key is used to sign dynamic signature data, which is then subject to terminal verification.

The card's public key is also used by the terminal during DDA verification. The RSA public/private key pair is unique to each IC card. This public/private key pair is generated by the issuer during the card personalization process and then written into the card's internal memory; the issuer also generates a public key certificate using the card's private key which is then also written into the card. When a terminal executes card DDA verification, it first utilizes an onboard IC Card Root CA public key certificate to verify the card's issuer public key certificate, then derive the issuer's public key from the issuer public key certificate. This issuer public key certificate is then used to verify the IC card public key certificate within the card. This IC card public key certificate is then used to derive the card's public key, which is then used to verify the signature of the DDA signature data of the card.

Composite dynamic data authorization (CDA) is an extension of DDA. CDA not only uses card private keys to sign dynamic transaction data but also to sign application cyphertext messages. This will guard against attacks on transaction data.

9.1.4 Other Keys

Aside from the keys used for the card transactions explained above, during the card personalization process issuers must also have secure transmission keys that are used to protect personalized data; see Section 10.9 for more details. Multi-application IC cards may also require additional key types.

9.2 Exporting Procedure Keys

All UICS debit/credit applications possess a certain number of main keys; IC card main keys are exported from different issuer main keys. IC card main keys include MK_{AC} , MK_{SMC} , and MK_{SMI} . The process keys SK_{AC} , SK_{SMC} , and SK_{SMI}

are exported by their corresponding IC card main keys: see Section 10.1 for details.

9.3 Key Length and Usage Cycle

The security of a ciphertext system increases with the length of the keys used. As hardware equipment is upgraded and the computing resources of attackers of cryptographic algorithms increases, the security of keys of a certain length decreases; these keys must be continuously replaced by longer keys.

It is suggested that double-length symmetrical 3DES keys be used, such as 112-bit keys plus an additional 16-bits of checksum values for a total of 128-bits. For Asymmetrical public/private key pairs, the largest public key length used by UICS debit/credit applications is 1984-bits.

The length of public keys is largely determined by IC Card Root CA. The length of a public key must balance the requirements of many factors, including transaction time, supporting devices, and security.

As attackers change and attacker resources grow, the shortest key length allowable for a given set of security requirements is subject to constant change. In 2005, an RSA public key with a length of 640-bits was successfully defeated by cryptographic researchers, therefore proving that 640-bit RSA public keys are capable of being defeated. The next target for researchers is a 704-bit RSA public key. Generally speaking, it is computationally-possible to defeat RSA public keys with a length of 512-bits.

With DDA's three-tier public key system, the security of the Root CA public/private key pair is by far the most important as the leak of this key pair would cause the collapse of the entire public/private key pair system. At the same time, verification of Root CA public key data only occurs in terminals, which have computational capabilities that far exceed those found in cards. For this reason, consideration for Root CA public/private key pairs should focus on maximum security. Public key modulus length and usage cycles should be strictly controlled under all possible situations.

The security of issuer public/private key pairs is more important than the security of card public/private keys pairs because if the latter becomes compromised then only the offline transactions of a single card are affected; card transaction risk control allows to reduce the risk of fake card transactions down to a relatively small level. At the same time, the computing power of cards is lower than that for terminals; as long as security requirements are within tolerances, selecting card public/private key lengths that are shorter than Root

CA public/private key lengths allows for an improvement in the efficiency of card signature operations.

The leakage of issuer private keys would cause very negative consequences for the operation of the issuer; it would allow either for the appearance of a large number of offline-verified fake cards or cause all of the SDA and DDA cards using the private signature of this issuer to become worthless. Therefore it is important that issuers use a longer public key modulus. The length of an issuer public key modulus does not affect the amount of time required for a card to conduct dynamic data signature operations. Verifying issuer public key certificates using Root CA public key certificates and verifying static data or card certificates using issuer public keys is accomplished by terminals, which have much higher computational power than cards have. For maximum security and computational efficiency, the issuer public key modulus length chosen should be the same as the length used by the Root CA public key modulus used to issue issuer certificates.

UnionPay will regularly evaluate and adjust the expiry dates of the financial IC card root CA public key. Latest public key proposals and expiry dates can be found on UnionPay official website.

According to UICS specifications and Financial IC Card Debit/Credit Application Root CA Public Key Specifications, card public key modulus length N_{IC} , issuer public key modulus length N_I , and Root CA public key modulus length N_{CA} should satisfy: $N_{IC} \leq N_I \leq N_{CA}$. 3 or $2^{16}+1$ can be chose for the public key indicator. This guide suggests that card public key modulus length N_{IC} should be no less than 1024 bits.

It is recommended that issuer public key modulus length (the key used to sign issuer public key certificates) is the same as the Root CA public key modulus length and only 3 are chosen for public key index. This is because selecting an issuer public key modulus length that is the same as the Root CA public/private key modulus length used to sign issuer public key certificates allows for the optimal balance of security and operational efficiency. Public key indexes of 3 or $2^{16}+1$ are not different in terms of security, but a public key modulus of 3 allows more vastly high operational efficiency.

Issuer symmetrical main keys should be updated annually. This does not mean that issuers must update the keys inside all issued cards, but that the issuer must be capable of managing different key forms. Should a given card and corresponding card key created by the export of this kind of key form become expired and not subject to dispute, then the issuer should destroy this issuer key form.

9.4 UICS Debit/Credit Application Key Summary

This section provides a summary of all of the keys used in UICS debit/credit application transaction processes. For a summary of keys, see Table 3.

Table 3: UICS Debit/Credit Application Key Summary

Name	Indicator	Type	Sharing Entities	Usage	Generation	Notes
IC Card Root CA Public Key	P_{CA}	RSA Public Key	Root CA, Issuers, Terminals	Verification of issuer public key certificate	Root CA	Public key (transmitted as self-signed certificate)
IC Card Root CA Private Key	S_{CA}	RSA Private Key	Not shared	Signature of issuer certificate	Root CA	Storage, used for Root CA encryptor
Issuer RSA Public/Private Key Pair	P_I S_I	RSA	Public keys: Root CA, Issuers, Cards Private keys: Not shared	Public Key: Verification of SDA, card certificate Private Key: SDA generation, signing of card certificate	Issuer	Issuer public key certificate issued by Root CA; written into card during card personalization
Card RSA Public/Private Key Pair	P_{IC} S_{IC}	RSA	Private key: issuers, cards Public keys: Cards (card certificate transmitted to terminal during transactions)	Private keys: DDA generation Public keys: DDA verification	Generated by issuer and written into card during personalization	Public/private key pair unique to each card; card public key certificate signed by issuer private key

Name	Indicator	Type	Sharing Entities	Usage	Generation	Notes
Issuer Application Ciphertext Main Key	IMK _{AC}	3DES	Issuers, Personalization/Authorization Systems	Exporting card application ciphertext main key MK _{AC}	Issuers	Shared by card personalization and authorization systems
Issuer Security Ciphertext Encryption Main Key	IMK _{SMC}	3DES	Issuers, Personalization/Authorization Systems	Exporting card security ciphertext main key MK _{SMC}	Issuers	Shared by card personalization and authorization systems
Issuer Security Ciphertext MAC Main Key	IMK _{SMI}	3DES	Issuers, Personalization/Authorization Systems	Exporting card security ciphertext MAC main key MK _{SMI}	Issuers	Shared by card personalization and authorization systems
Card Application Ciphertext Main Key	MK _{AC}	3DES	Issuers, Cards	Export card application ciphertext process key SK _{AC}	Exported using IMK _{AC} by issuer and written into card during personalization	Unique to each card
Card Security Ciphertext Encryption Main Key	MK _{SMC}	3DES	Issuers, Cards	Exporting card security ciphertext encryption process key	Exported using IMK _{SMC} by issuer and written into card during personalization	Unique to each card

Name	Indicator	Type	Sharing Entities	Usage	Generation	Notes
Card Security Ciphertext MAC Main Key	MK_{SMI}	3DES	Issuers, Cards	Exporting card security ciphertext MAC process key	Exported using IMK_{SMI} by issuer and written into card during personalization	Unique to each card
Card Application Ciphertext Process Keys	SK_{AC}	3DES	Only exists for short period during transaction processes; shared by card and authorization system	Generating application ciphertext	MK_{AC} used by Card (authorization system) and ATC export	Used only for single transactions and then discarded
Card Security Ciphertext Encryption Process Key	SK_{SMC}	3DES	Only exists for short period during transaction processes; shared by card and authorization system	Encrypts and decrypts security ciphertext	MK_{SMC} used by Card (authorization system) and ATC export	Used only once for script processing and then discarded
Card Security Ciphertext MAC Process Key	SK_{SMI}	3DES	Only exists for short period during transaction processes; shared by card and authorization system	Computes security ciphertext and verifies MAC	MK_{SMI} used by Card (authorization system) and ATC export	Used only once for script processing and then discarded

10 UICS Debit/Credit Application Key Cryptographic Algorithms

This section provides the cryptographic algorithms and associated keys used by UICS debit/credit applications.

10.1 Card Main Key and Process Key Generation Cryptographic Algorithms

10.1.1 Card Main Key Generation Cryptographic Algorithms

Issuers possess three main keys: application ciphertext issuer main key IMK_{AC} , secure ciphertext encryption issuer main key IMK_{SMC} , and secure ciphertext MAC issuer main key IMK_{SMI} .

Cards also possess three main keys: application ciphertext card main key MK_{AC} , secure ciphertext encryption card main key MK_{SMC} , and secure ciphertext MAC card main key MK_{SMI} .

Card main keys are generated using ciphertext generated by issuer main keys by utilizing 3DES cryptographic algorithms to encrypt the link values of the card's PAN and PAN sequence numbers.

$$MK_{AC} = 3DES(IMK_{AC})[Input] || 3DES(IMK_{AC})[Input \oplus \text{'FF FF FF FF FF FF FF FF'}]$$

$$MK_{SMC} = 3DES(IMK_{SMC})[Input] || 3DES(IMK_{SMC})[Input \oplus \text{'FF FF FF FF FF FF FF FF'}]$$

$$MK_{SMI} = 3DES(IMK_{SMI})[Input] || 3DES(IMK_{SMI})[Input \oplus \text{'FF FF FF FF FF FF FF FF'}]$$

Input here are the card's PAN and PAN sequence numbers; when the linking of PAN and PAN sequence numbers is 8 bytes (16 numbers) long, then the numerical form of this sequence's link Y will be the input; If the connecting of PAN and Pan sequence numbers is less than 16 bytes, then align to the right and add hexadecimal 0s to create an 8 byte form of Y; if the connecting of PAN and PAN sequence numbers has at least 16 digits, then Y=the leftmost 16 of these digits.

3DES is a double-length, triple iteration DES calculation: the key is (Key A||Key B). Key A and Key B are 56-bit single DES keys; the triple iteration DES calculation of 3DES is as follows:

1. Utilize Key A to perform single DES calculation on inputted data,
2. Use Key B to perform a single DES decryption calculation on the results of the above operation,
3. Use Key A to perform a single DES encryption calculation on the results of the above operation.

10.1.2 Card Main Key Process Key Generation Cryptographic Algorithms

All card online transactions use process keys exported from card main keys to calculate online transaction application ciphertexts and online transaction security ciphertexts. Application ciphertexts are calculated using the card application process key SK_{AC} , security ciphertext encryption uses card security ciphertext encryption process key SK_{SMC} , and security ciphertext MAC is calculated using card security ciphertext MAC process key SK_{SMI} .

Card process keys SK_{AC} , SK_{SMC} , and SK_{SMI} are exported by card main keys MK_{AC} , MK_{SMC} , and MK_{SMI} respectively. Process key are the ciphertexts created by the encryption conversion of the card's ATC values using that card's corresponding main keys. The conversion of card ATC values is as follows:

$$X := '00' || '00' || '00' || '00' || '00' || '00' || (ATC)$$

$$Y := '00' || '00' || '00' || '00' || '00' || '00' || (ATC \oplus 'FFFF')$$

$$\text{Process Key A} := 3DES(MK)[X]$$

$$\text{Process Key B} := 3DES(MK)[Y]$$

Final Process Key: Process Key A || Process Key B. "||" indicates a data connection. Therefore:

$$SK_{AC} = 3DES(MK_{AC})['00' || '00' || '00' || '00' || '00' || '00' || ATC] || 3DES(MK_{AC})['00'$$

$$|| '00' || '00' || '00' || '00' || '00' || (ATC \oplus 'FFFF')]$$

$$SK_{SMC} = 3DES(MK_{SMC})['00' || '00' || '00' || '00' || '00' || '00' || ATC] || 3DES(MK_{SMC})['00'$$

$$|| '00' || '00' || '00' || '00' || '00' || (ATC \oplus 'FFFF')]$$

$$SK_{SMI} = 3DES(MK_{SMI})['00' || '00' || '00' || '00' || '00' || '00' || ATC] || 3DES(MK_{SMI})['00'$$

$$\| '00' \| '00' \| '00' \| '00' \| '00' \| (ATC \oplus 'FFFF') \]$$

10.2 Card and Issuer Online Application Ciphertext Processing

The card completing a card online transaction must use cryptographic algorithms to generate TC, AAC, and ARQC; Issuers must generate ARPC.

10.2.1 Online Application Ciphertext Generation Cryptographic Algorithms

The online application ciphertexts ARQC, AAC, and TC are generated using cryptographic algorithms in accordance with ISO/IEC 9797-1 Specifications/Algorithm 3, a CBC-type 64-value derived cryptographically calculated MAC algorithm; the basic segment encryption algorithm is the standard DES algorithm.

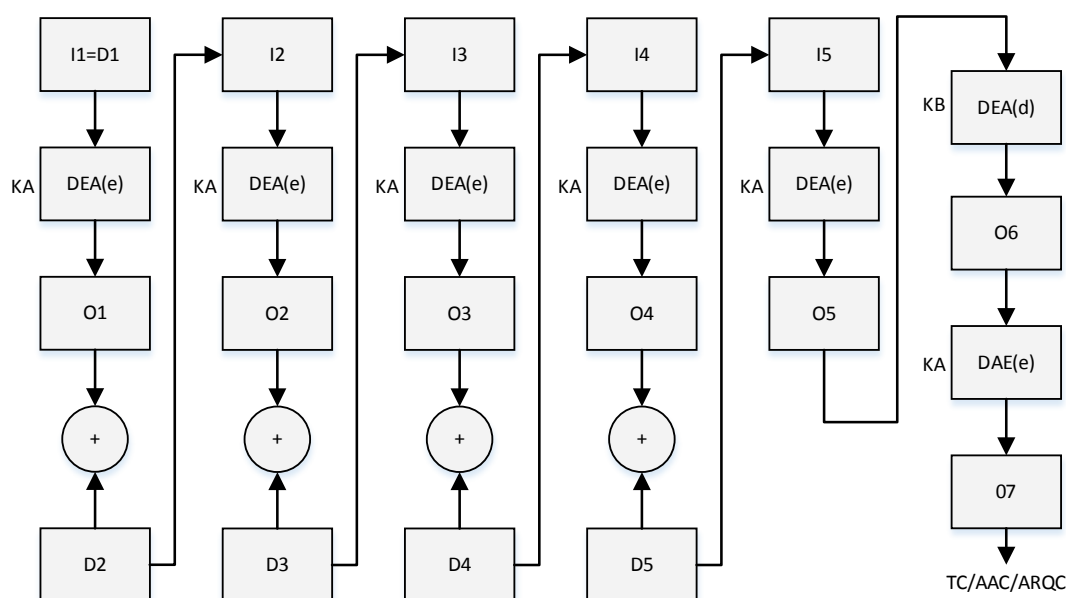
$$ARQC := MAC(SK_{AC})[Input]$$

$$TC := MAC(SK_{AC})[Input]$$

$$AAC := MAC(SK_{AC})[Input]$$

The MAC algorithm consists of the following:

Algorithm input should be derived into 8 byte data segments: $D_1, D_2, \dots, D_n$; K_A is Key A, K_B is Key B; The computation should be conducted in the following manner:



Explanation:

I = Input

D = Data segment

DEA(e)= data encryption algorithm (encryption mode)

KA = Key A

DEA(d)= data encryption algorithm (decryption mode)

KB = Key B

O = Output

+ = XOR

Figure 3 Online Application Ciphertext MAX Algorithm

ARPC calculations utilize the 3DES algorithms as follows:

ARPC:=3DES(SK_{AC})[Input]

10.2.2 Online Application Ciphertext Generation

10.2.2.1 ARQC, TC, and AAC Generation

Online application ciphertexts ARQC, TC, and AAC are computed using the following algorithms:

ARQC:=MAC(SK_{AC})[Input]

TC:=MAC(SK_{AC})[Input]

AAC:=MAC(SK_{AC})[Input]

The algorithm input is created as follows:

All inputs used for the calculation of ARQC, AAC, and TC are arranged in accordance with the data of Table 4. (See UICS Card Specifications):

Table 4: TC/AAC/ARQC Data Element Sequence

Data Element	Data Transmitted from Terminal	Transaction Certificate (TC) Hash Sequence	Onboard Card Data
Authorized Amount	● □	● □	

Data Element	Data Transmitted from Terminal	Transaction Certificate (TC) Hash Sequence	Onboard Card Data
Other Amounts	● ☐	● ☐	
Terminal Country Code	● ☐	● ☐	
Terminal Verification Result	● ☐	● ☐	
Transaction Currency Code	● ☐	● ☐	
Transaction Date	● ☐	● ☐	
Transaction Type	● ☐	● ☐	
Unpredictable Number	● ☐	● ☐	
Application Interchange Profile (AIP)			● ☐
Application Transaction Counter (ATC)			● ☐
Card Verification Result (CVR)			● ☐

The link values of the data in Table 4 according to their original sequence are the inputs for MAC algorithms (if these values are not in the form of 8 byte whole multiples, add "80" and a sufficient number of "00" until they become so.)

MAC algorithms are in accordance with ISO/IEC 9797-1 Specifications/Algorithm 3 and use a CBC-type 64-value derived cryptography calculation as illustrated below:

Segment algorithm input into 8 byte data blocks: $X_1, X_2, \dots, X_K$. In accordance with ISO/IEC 9797-1 algorithm 3: $H_{K+1} := \text{ALG}(K_{SL})[\text{ALG}^{-1}(K_{SR})[H_K]]$. Here, the initial value of $H_0 = ('00' || '00' || '00' || '00' || '00' || '00' || '00' || '00')$. (See UICS Security Specifications, Section 9.1.2.1).

10.2.2.2 ARPC Generation

$ARPC = 3DES(SK_{AC})[Input]$, whereas the input is composed in the following manner:

Take the XOR of an application ciphertext AC (generally either ARQC or AAC) and the external response code ARC (this external response code should first be aligned to the left and have six “00” bytes added to the right); the XOR result value is the input. (See UICS Card Specifications D.1). namely:

$$ARPC = 3DES(SK_{AC})[Input] = 3DES(SK_{AC})[AC \oplus (ARC \parallel ('00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00'))]$$

10.3 Card and Issuer Secure Message Processing

Security message processing is used for processing issuer script commands and can be divided into encryption processing and integrity processing. UICS debit/credit applications touch upon the following script commands: application locking, application unlocking, card locking, PIN changes/unlocking, data configuration, and record modification. The script process for PIN changes/unlocking requires PIN encryption while the remaining scripts only require computing MAC values.

10.3.1 Secure Message Process Keys

Security message process key calculations are all in accordance with Section 9.2, as shown below:

The calculation of secure message encryption process key SK_{SMC} uses card secure message encryption key MK_{SMC} , while the calculation of secure message MAC process key SK_{SMI} uses card secure message MAC main key MK_{SMI} .

10.3.2 Secure Message Cryptographic Algorithm

The secure message encryption algorithm is as follows: ECB-mode 3DES (SK_{SMC})[Input]; See UICS Security Specifications, Section 9.1.1 as well as the following section.

10.3.2.1 Secure Message Encryption Algorithm

Data encryption uses a derived (64-digit, 8-byte) encryption algorithm following an electronic code book (ECB) model.

Using encryption process key K_s for the encryption of a message MSG of any length uses the following steps:

1. Pad and Segment

If the length of the message MSG is not a whole multiple of the length of its segments, then add one "80" byte to the right-side of the message and then add as less "00" as necessary such that the resulting message length $MSG := (MSG \parallel '80' \parallel '00' \parallel '00' \parallel \dots \parallel '00')$ is a whole multiple of the segment length.

If the length of message MSG is a whole multiple of the segment length then padding in additional data is not necessary.

Before encryption can be carried out on the data it must first be formatted into the following data block configurations:

- 1) Length of plaintext data, not including padded characters
- 2) Plaintext data
- 3) Padding characters (in accordance with method described above); then break MSG up into 8 byte or 16-byte segments $X_1, X_2, \dots, X_K$.

2. Ciphertext Calculation

ECB Mode

Use encryption process code K_s in ECB mode to apply derived encryption to data blocks $X_1, X_2, \dots, X_K$, encrypting them into segment length blocks $Y_1, Y_2, \dots, Y_K$.

Therefore, when $i=1, 2, \dots, K$, compute in the following manner:

$$Y_i := \text{ALG}(K_s)[X_i].$$

10.3.2.2 Secure Message Encryption Algorithm Inputs

Secure message encryption is only used when an issuer is generating a PIN or modifying script commands; during these times the new PIN must be encrypted.

If not using the current PIN modification/PIN encryption data production process, then follow the following steps:

1. Generate 8-byte PIN data block D3:

a) Generate an 8-byte data block D1:

Byte 1		Byte 2		Byte 3		Byte 4		Byte 5	Byte 6	Byte 7	Byte 8
0	0	0	0	0	0	0	0	Last 4 bytes of righthand-end of secure message encryption process key A			

b) Generate an 8-byte data block D2:

Byte 1		Byte 2		Byte 3		Byte 4		Byte 5		Byte 6		Byte 7		Byte 8	
0	N	P	P	P	P	P/F	P/F	P/F	P/F	P/F	P/F	P/F	P/F	F	F

N: Number of (hexadecimal) digits of new PIN

P: New PIN value, length of 4-12 numbers (2-6 bytes)

c) Derive D3 by XOR of D1 and D2

2. Use current PIN to generate 8-byte data block D4:

Byte 1		Byte 2		Byte 3		Byte 4		Byte 5		Byte 6		Byte 7		Byte 8	
P	P	P	P	P/0	P/0	P/0	P/0	P/0	P/0	P/0	P/0	0	0	0	0

3. Derive D using XOR of data blocks D3 and D4.

The D described above is the secure message encryption input. If using the current PIN modification/PIN encryption data production process, then follow the following steps:

1. Generate an 8-byte PIN data block D3:

a) Generate an 8-byte data block D1:

Byte 1		Byte 2		Byte 3		Byte 4		Byte 5		Byte 6		Byte 7		Byte 8	
--------	--	--------	--	--------	--	--------	--	--------	--	--------	--	--------	--	--------	--

0	0	0	0	0	0	0	0	Last 4 bytes of righthand-end of secure message encryption process key A
---	---	---	---	---	---	---	---	--

b) Generate a second 8-byte data block D2:

N: Number of (hexadecimal) digits of new PIN

P: New PIN value, length of 4-12 numbers (2-6 bytes)

c) Derive D3 by XOR of D1 and D2

2. Use current PIN to generate 8-byte data block D4:

Byte 1		Byte 2		Byte 3		Byte 4		Byte 5		Byte 6		Byte 7		Byte 8	
P	P	P	P	P/0	P/0	P/0	P/0	P/0	P/0	P/0	P/0	0	0	0	0

3. Derive D using XOR of data blocks D3 and D4.

The D described above is the secure message encryption input.

10.3.2.3 Secure Message MAC Algorithm

The secure message MAC algorithm is the following: $MAC = MAC4(SK_{SMI}[Input])$; MAC4 represents the first 4 bytes of MAC. For MAC computation specifics see UICS Section 11.1.2.1; for input composition see UICS Appendix C.

The length s of MAC is 4 bytes.

The calculation of a 4-byte MAC should be in accordance with ISO/IEC9797-1 specifications and use a CBC mode, 64-value derived encryption algorithm. The following steps are used to calculate a MAC value S using MAC process key SK_{SMI} on a message MSG of any length:

Pad and segment: The message MSG should be padded in accordance with Mode 2 of ISO/IEC9797-1 by first forcibly adding one "80" byte to the left side of the MSG and then adding as less "00" bytes as needed such that the length of the message $MSG := (MSG || '80' || '00' || '00' || \dots || '00')$ is a whole integer multiple of 8-bytes. Afterwards the MSG should be derived into 8-byte blocks, i.e. $X_1, X_2, \dots, X_K$.

The MAC process key is $SK_{SMI} = (K_{SL} \parallel K_{SR})$, with K_{SL} being either the left half of SK_{SMI} or key A; K_{SR} is either the right half of SK_{SMI} or key B.

Use the far-right block K_{SL} of MAC process key and conduct CBC mode derived encryption to process it into 8-byte blocks $X_1, X_2, \dots, X_K$:

$$H_i := \text{ALG}(K_{SL})[X_i \oplus H_{i-1}], \text{ here } i = 1, 2, \dots, K.$$

The starting value of $H_0 := ('00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00')$.

In accordance with ISO/IEC 9797-1 Algorithm 3: $H_{K+1} := \text{ALG}(K_{SL})[\text{ALG}_{-1}(K_{SR})[H_K]]$.

MAC value S is equivalent to the 4 highest value bytes of H_{K+1} .

Here, ALG indicates a standard DES computation.

10.3.2.4 Secure Message MAC Algorithm Input

Used for issuer script processing of secure message functions such as application locking, application unlocking, card locking, PIN modification/unlocking, data setup, and record modifications. All of these secure messages require MAC calculations.

Commands require that encrypted data (such as PIN modification) calculate MAC after encryption. MAC is calculated using symmetrical key algorithms with the following steps:

1. The starting value is 8-bytes of "0". (This step can be ignored)
2. A data block D is obtained using the following data sequence:
 - CLA, INS, P1, P2, Lc (Lc length includes MAC length)
 - ATC (for issuer script processing, this ATC is sent with the request message)
 - Application ciphertext (this application ciphertext is usually either ARQC or AAC and is sent with the request message)
 - Command data field plaintext or ciphertext (if it exists)
3. Segment the data block D shown above into 8-byte data blocks $D_1, D_2, D_3, \dots$ with the byte length of the last data block being between 1 to 8.

4. If the length of the last data block is 8-bytes, then add an 8-byte data block to the end, i.e.: "80 00 00 00 00 00 00 00" and go to step 5. If the length of the last data block is less than 8-bytes then add "80" to the end. Add "00" to the end of the data block until its length is 8-bytes.
5. Use MAC process key SK_{SMI} and encrypt the data blocks.
6. The calculated MAC result will be 8 bytes long; select the first 4 bytes starting from the left-hand side.

10.4 Offline Data Authentication Processes

Card offline data authentication can be divided into static data authentication, dynamic data authentication and composite dynamic data authentication/application ciphertext generation.

10.4.1 Financial IC Card Debit/Credit Application Public Key Authentication System Overview

The Financial IC Card Debit/Credit Application Public Key Authentication System is in accordance with the *Financial Industry Integrated Circuit (IC) Card Specifications* and provides public key authentication services for all financial IC card debit/credit applications that abide by this standard. At the same time this system also provides services for other financial IC card data authentication (such as data authentication for member institutions).

In accordance with the *Financial Industry Integrated Circuit (IC) Card Specifications* and financial IC card debit/credit application Root CA public key authentication specifications, the Financial IC Card Debit/Credit Application Authentication system includes Root CA, all issuer CA and associated UnionPay standard financial IC cards, and acquirer institution ATMs and/or POS. The Financial IC Card Debit/Credit Application Public Key Authentication System utilizes public key technology to generate and authentication financial IC card static data, dynamic data, and composite data so as to provide the highest level of security for financial IC card transaction authentication services.

- Root CA is responsible for the generation of Root CA public keys, the management of Root CA public key information, issuing issuer CA public key certificates and securely transmitting Root CA public key information to acquirer institutions. It is the apex of trust within the financial IC card debit/credit application public key authentication system.

- All issuer CA is a subset CA of Root CA and is responsible for the generation of issuer CA public/private key pairs, sending public key certificate applications to Root CA, and managing one's own public key certificates. Additionally, issuer CA communicates with card issuance systems for the signing of IC card static application data and static data authentication (SDA), generating IC card public/private key pairs, issuing IC card public key certificates, and conducting dynamic data authentication (DDA). At the same time issuer CA also writes its own public key certificates, IC card public key certificates, IC card private keys, RID, and Root CA public key indices into cards.
- Acquirer institutions are responsible for building terminal management systems, sharing Root CA keys with participating terminals (such as POS/ATMs), and conducts long-range equipment management, status monitoring, and information monitoring (including code/parameter downloads) on those terminals.

Within the standard card payment system of *Financial Industry Integrated Circuit (IC) Card Specifications*, IC cards store IC card public key certificates and IC card private keys (or static data) as well as issuer public key certificates, RID, and Root CA public key indices; participating terminals store Root CA public keys, public key indices, and associated RID.

Within the payment system, participating terminals conduct IC card data authentication using IC card application data signatures. This process is as follows: first, the RID, IC card public key certificate, issuer public key certificate, and Root CA public key indices are read from the card. RID and Root CA indices are used to determine a specific Root CA public key; this Root CA public key is then used to authenticate the issuer public key certificate, which in turn is used to authenticate the IC card public key certificate. The IC card public key certificate is then used to authenticate the IC card's dynamic data signatures or the issuer public key may be used to directly authenticate the IC card's static data signatures. The entire IC card data authentication process is conducted offline. See Figure 4:

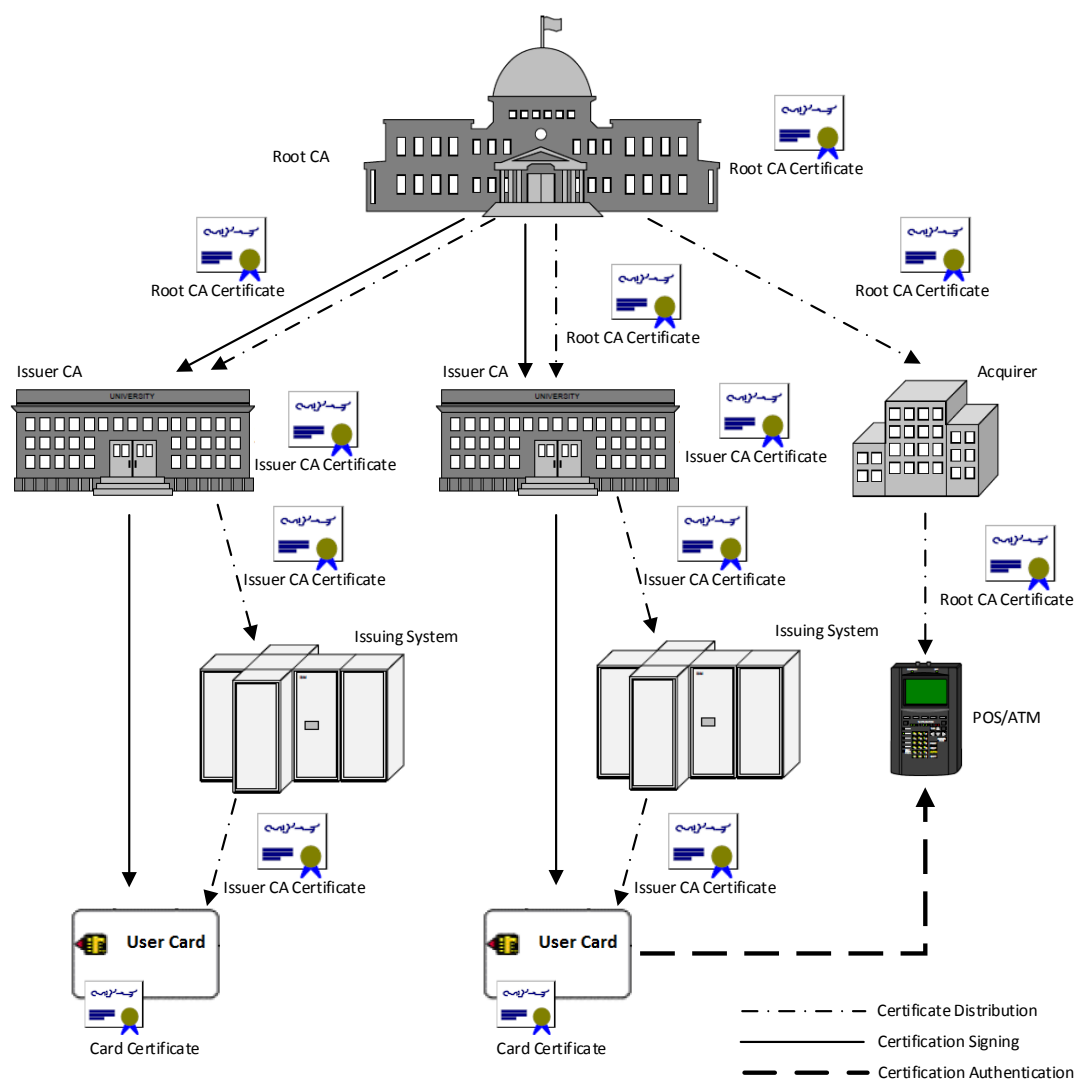


Figure 4 IC Card Root CA Public Key Authentication System

10.4.2 Offline Authentication Process Keys

1. The generation and authentication of static signature data uses the following keys:
 - IC card Root CA public/private keys pairs used for issuing issuer public key certificates
 - Issuer public/private key pairs used for issuing card static data
2. The generation and authentication of dynamic signature data uses the following keys:

- IC card Root CA public/private keys used for issuing issuer public key certificates
 - Issuer public/private key pairs used for issuing card public key certificates
 - Issuer public/private key pairs issuing card dynamic data
3. Aside from the keys described above used for dynamic data authentication, keys used for composite dynamic data authentication/application ciphertext generation and authentication processes also include card application ciphertext process keys; see Section. 10.1.

10.4.3 Offline Authentication Process Cryptographic Algorithms

RSA algorithms utilizing SHA-1 hash functions are used for offline authentication signature algorithms; see Section 12 of UICS Security Specifications.

10.4.4 Offline Authentication Generation and Verification Processes

10.4.4.1 Static Data Authentication

Static data authorization is completed by the verification of a card's static data signature by a terminal. The objective of this authorization is to determine the legality of the static data stored aboard a UnionPay standard IC card; this process can determine whether or not the issuer data stored aboard the card has been modified without authorization and can accurately detect the authenticity of UnionPay IC card static data. See Figure 5.

The entire UnionPay standard IC card static data authorization process is as follows:

- 1) The key management system of the issuer generates issuer public/private key pair P_I and S_I and then transmits P_I to Root CA;
- 2) Root CA uses its own private key S_{CA} to sign issuer public key P_I , generates an issuer public certificate and then sends it to the issuer key management system together with a Root CA public key certificate (including RID and Root CA public key indices);
- 3) The issuer key management systems use issuer private key S_I to electronically sign card static data and then transmit the signature result,

issuer public key certificate, RID, and Root CA indices to the card issuance system;

- 4) During the card personalization process, the card issuance system writes the static electronic signatures, issuer public key certificate, RID, and Root CA indices into the card.
- 5) Root CA sends its public key P_{CA} , RID, Root CA public key index, and other related information to the terminal management system by way of the acquirer;
- 6) The terminal management system of the acquirer downloads Root CA public key P_{CA} , RID, and Root CA public key index into the terminal;
- 7) The offline static data authentication process for a UnionPay standard IC card is as follows:
 - The terminal reads the issuer public key certificate and signed data from the card and uses the Root CA public key indices and RID to find the Root CA public key P_{CA} to recover and authenticate issuer public key P_I ;
 - The terminal uses the recovered issuer public key P_I to verify the effectiveness of the card's signature data.

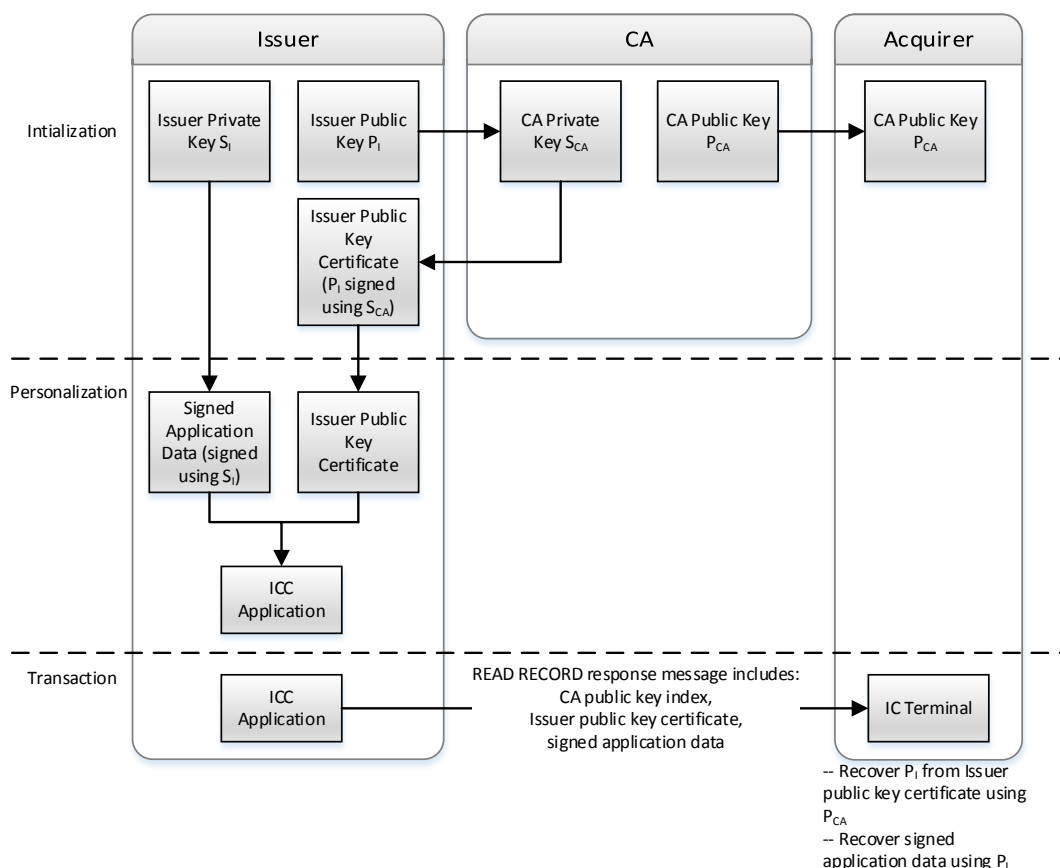


Figure 5 SDA Certificates and Public Key System Structure

10.4.4.2 Dynamic Data Authentication

During the dynamic data authorization (DDA) process, the terminal verifies card static data and dynamic transaction data signatures. DDA can determine whether or not the card's application data has been illegally altered after personalization, though more importantly it also determines the veracity of the card. This prevents illegal copies and fake cards.

DDA works for standard dynamic data authorization as well as composite dynamic data authorization/application ciphertext generation (CDA). UnionPay standard IC card dynamic data authorizations are illustrated in Figure 6.

Using this method, the UnionPay standard IC card takes dynamic transaction data and terminal data identified by the dynamic data object list (DDOL) to generate an electronic signature (See China Financial Integrated Circuit (IC) Card Specifications).

UnionPay standard IC card standard dynamic data authorization occurs using the following process:

- 1) The key management system of the issuer generates issuer public/private key pair P_I and S_I and then transmits P_I to Root CA;
- 2) Root CA uses its own private key S_{CA} to sign issuer public key P_I , generates an issuer public certificate and then sends it to the issuer key management system together with a Root CA public key certificate (including RID and Root CA public key indices);
- 3) The issuer key management system generates a public/private key pair S_{ICC} and P_{ICC} for each UnionPay Standard IC card then uses an issuer private key S_I to electronically sign the IC card public key P_{ICC} , thus creating an IC card public key certificate;
- 4) The issuer key management key transmits the issuer public key certificate, IC card public key certificate, IC card private key, RID, and Root CA public key indices to the card issuance system;
- 5) The issuer key management key transmits the issuer public key certificate, IC card public key certificate, IC card private key, RID, and Root CA public into the card during the card personalization process;
- 6) Root CA sends its public key P_{CA} , RID, Root CA public key index, and other related information to the terminal management system by way of the acquirer institution;
- 7) The terminal management system of the participating institution downloads Root CA public key P_{CA} , RID, and Root CA public key index into the terminal;
- 8) The offline dynamic data authentication process for a UnionPay standard IC card is as follows:
 - The terminal reads the issuer public key certificate, IC card public key certificate, RID, and Root CA public key index from the card and then uses RID and the Root CA public key index to locate Root CA public key P_{CA} ; this public key is then used to recover and verify issuer public key P_I ; this issuer public key is then used to recover and verify IC public key P_{ICC} ;
 - The terminal sends an internal authentication commands (INTERNALAUTHENTICATE) (see China Financial Integrated Circuit (IC) Card Specifications) to the IC card and requests a dynamic signature; the card connects the terminal data and IC card transaction dynamic data within

the internal authentication command and then uses the IC card private key S_{ICC} to sign the connected data and return it to the terminal;

- The terminal uses IC card public key P_{ICC} to verify the electronic signature described above.

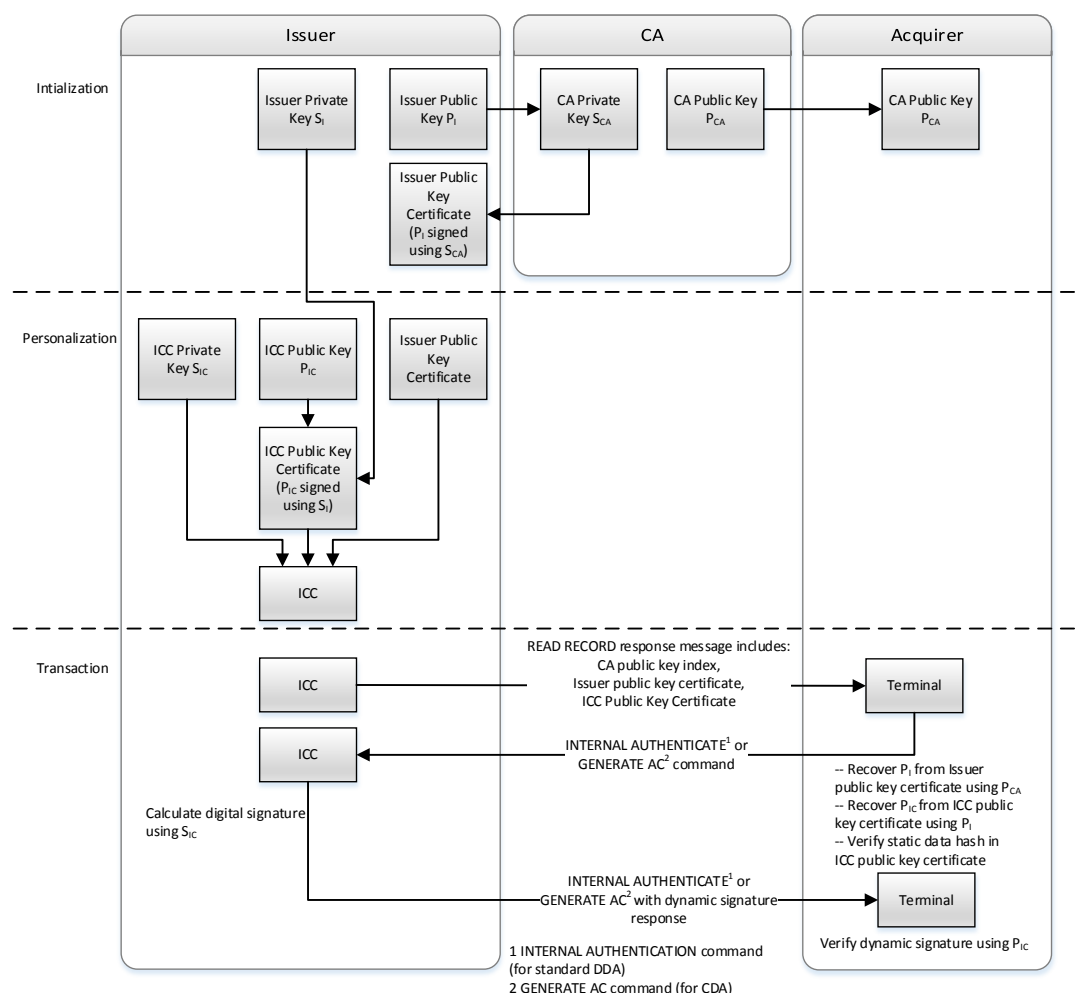


Figure 6 DDA Certificates and Public Key System Structure

10.4.4.3 Composite Dynamic Data Authorization/Application Ciphertext Generation

This method is executed after the first application ciphertext command request is sent (Financial Integrated Circuit (IC) Card Specifications). UnionPay Standard IC cards use card data (including application ciphertext and terminal data) to generate an electronic signature.

The UnionPay Standard IC Card composite dynamic data authorization/application ciphertext generation process is as follows:

- 1) The key management system of the issuer generates issuer public/private key pair P_I and S_I and then transmits P_I to Root CA;
- 2) Root CA uses its own private key S_{CA} to sign issuer public key P_I , generates an issuer public certificate and then sends it to the issuer key management system together with a Root CA public key certificate (including RID and Root CA public key indices);
- 3) The issuer key management system generates a public/private key pair S_{ICC} and P_{ICC} for each UnionPay Standard IC card then then uses the issuer private key S_I to electronically sign the IC card public key P_{ICC} , thus generating an IC card public key certificate;
- 4) The issuer key management key transmits the issuer public key certificate, IC card public key certificate, IC card private key, RID, and Root CA public key indices to the card issuance system;
- 5) The issuer key management key transmits the issuer public key certificate, IC card public key certificate, IC card private key, RID, and Root CA public into the card during the card personalization process;
- 6) Root CA sends its public key P_{CA} , RID, Root CA public key index, and other related information to the terminal management system by way of the participating institution;
- 7) The terminal management system of the participating institution downloads Root CA public key P_{CA} , RID, and Root CA public key index into the terminal;
- 8) The offline composite dynamic data authentication process for a UnionPay standard IC card is as follows:
 - The terminal reads the issuer public key certificate, IC card public key certificate, RID, and Root CA public key index from the IC card.
 - The terminal uses RID and Root CA indices to locate the Root CA public key P_{CA} ; it then uses Root CA public key P_{CA} to verify the issuer public key certificate signature and recover issuer public key P_I .
 - The terminal uses issuer public key P_I to verify the signature of the IC card public key certificate and recover IC card public key P_{ICC} .

- The terminal generates an unpredictable number and then transmits it to the IC card along with associated data.
- The IC card uses its private key S_{ICC} to electronically sign received terminal data (including unpredictable numbers and transaction data) and other IC card data (including TC/ARQC) and transmit it them to the terminal.
- The terminal uses IC card public key P_{ICC} to verify the signed information transmitted by the IC card.

For more information on card offline authorization processing, see UICS Security Specifications, Chapter 5 and Financial IC Card Debit/Credit Application Root CA Public Key Authorization Specifications.

10.5 IC Card CA Public Key Files

Refer to chapter 6 of UICS *Part VII UnionPay Integrated Circuit Card Debit Credit Application Root CA Public Key Authentication Specification - Technical Specification*.

10.6 Issuer Public Key Input Files

Refer to chapter 8 of UICS *Part VII UnionPay Integrated Circuit Card Debit Credit Application Root CA Public Key Authentication Specification - Technical Specification*.

10.7 Issuer Public Key Output Files

Refer to chapter 10 of UICS *Part VII UnionPay Integrated Circuit Card Debit Credit Application Root CA Public Key Authentication Specification - Technical Specification*.

10.8 Card Certificates and Signed Static Data Generation

10.8.1 Issuer-Issued Card Certificates

Issuers must sign card public key certificates during the issuance process for DDA/CDA cards. Issuer CA-signed card public key certificates are created in the form of card public key files.

Card public key files come in the format: AAAAAAAAAA.ICNNNNNN. "AAAAAAAAAA" indicates the card account number (PAN); IC is a fixed

value ('IC') indicating the card; NNNNNN is the issuer CA public key certificate sequence number used to issue the card public key certificate.

10.8.1.1 Card Public Key File Content

Card public key files are constructed from unsigned card public key output extensions and signed card certificates.

Table 5: Card Public Key Files

Field Name	Length (No. of bytes)
Unsigned Card Public Key Output Extensions	$18 + e_{IC}$ If $N_{IC} \leq N_I + 42$ $60 + N_{IC} - N_I + e_{IC}$ If $N_{IC} > N_I + 42$ Here, e_{IC} , N_{IC} , and N_I represents card public key byte count, card public key modulus length byte count, and the public key modulus length byte count used by Issuer CA to generate the card public key certificate
Signed Card Public Key Certificate	N_I

10.8.1.2 Unsigned Card Public Key Output Expansions

Unsigned card public key output extensions are the first part of card public key output files and have the form illustrated in Table 6. This public key output extension supplies card public key certificate information.

Table 6: Unsigned Card Public Key Output Expansions

Field Name	Length (No. of bytes)	Description	Code Format
Record Header	1	Hexadecimal '26'	b
Card Main Account Number(PAN)	10	Main account number (pad from right side with hexadecimal 'F')	cn 20

Field Name	Length (No. of bytes)	Description	Code Format
Certificate Sequence Number	3	A unique binary card certificate identifier allocated by Issuer CA	b
Certificate Expiry Date	2	Month and year (MMYY); the certificate expires on the last day of the month	n 4
Card Public Key Remainder Length	1	Length of Card Public Key Modulus Remainder	b
Card Public key Remainder	0 and greatest value of $(N_{IC} - N_I + 42)$	This section only exists during $N_{IC} > N_I - 42$ and is created by the lowest byte value $N_{IC} - N_I + 42$ of card public key (N_{IC})	b
Card Public Key Indicator Length	1	Hexadecimal-represented length of card public key indicator (byte count eIC)	b
Card Public Key Indicator	eIC	Card public key indicator, represented by 3 or 65537; stored hexadecimal as '03' or '01 00 01'	b

10.8.1.3 Signed Card Public Key Certificates

Signed card public key certificates are the second part of card public key files; they are signed and generated by Issuer CA using corresponding Issuer CA private keys on the card public key data displayed in Table 7.

Table 7: Card Public Key Certificate Data

Field Name	Length (No. of bytes)	Description	Code Format
Recovered Data Header	1	Hexadecimal value '6A'	b
Certificate Format	1	Hexadecimal value '04'	b
Application Main Account Number	10	Main account number (pad from right side with hexadecimal 'F'))	cn 20

Field Name	Length (No. of bytes)	Description	Code Format
Certificate Expiry Date	2	Month and year (MMYY); the certificate expires on the last day of the month	n4
Certificate Sequence Number	3	A unique binary certificate identifier allocated by the Issuer	b
Hash Algorithm Identifier	1	Used to identify hash algorithm used to generate electronic signature hash values	b
IC Card Public Key Algorithm Identifier	1	Used to electronic signature algorithm of IC card public keys	b
Card Public Key Length	1	Identifies byte length of IC card public key modulus	b
IC Card Public Key Indicator Length	1	Identifies byte length of IC card public key indicator	b
IC Card Public Key or Left-End Byte Segment of IC Card Public Key	$N_I - 42$	If $N_{IC} \leq N_I - 42$, the this segment—including the $N_I - 42 - N_{IC}$ value padded on the right with the bytes 'BB'—a complete IC card public key. If $N_{IC} > N_I - 42$, then this section includes the highest byte $N_I - 42$ of the IC card public key	b
Hash Result	20	Hash values for IC card public keys and associated information	b
Recovered Data End	1	Hexadecimal value 'BC'	b

Note: Hash results are created by first connecting Table 7's second to tenth data elements (i.e. from certificate format to the left-end bytes of IC card public keys) together in sequence from left to right; IC card public key remainders (if present) and IC card public key indicators are then added to the end of this data string, followed by any static data required according to Financial Integrated Circuit (IC) Card Specifications; finally, this string is then used to compute a hash value.

10.8.1.4 Hash Calculations of Card Public Key and Associated Information

Note: Hash results are created by first connecting Table 7's second to tenth data elements (i.e. from certificate format to the left-end bytes of IC card public keys) together in sequence from left to right; IC card public key remainders (if present) and IC card public key indicators are then added to the end of this data string, followed by any static data authorization requirements according to China Financial Integrated Circuit (IC) Card Specifications; finally, this string is then used to compute a hash value. See Table 8 for an explanation of the authorization requirements of Financial Integrated Circuit (IC) Card Specifications:

Table 8: Static Application Data Requiring Signatures

Field Name	Length (No. of bytes)	Description	Code Format
Application Interchange Profile (AIP)	2	Explains card functions supported by the application.	b
Application Effective Date	3	Effective data of card applications.	n
Application Expiry Date	3	Expiry date of card application	n
Application Account Number	10	PAN, if insufficient pad with "F" from right	cn
Application Main Account Number Sequence Number	1	Used to indicate different applications within a given card account.	n
Application Usage Control (AUC)	2	Explains issuer limits of given applications, including usage areas, services, etc. Used to supply more flexible cards with service controls (similar to codes).	b

Field Name	Length (No. of bytes)	Description	Code Format
Cardholder Verification Method List	12	Lists all cardholder verification methods supported by card applications in order of superiority. Note: An application may have many different CVM lists, such as one for domestic transactions and another for international transactions.	b
Issuer Activity Code (IAC)——Default	5	Sets conditions for when an online request for a transaction is made but cannot be completed by the terminal and is refused.	b
Issuer Activity Code (IAC)——Refusal	5	Sets conditions for when a transaction is not online and is directly refused.	b
Issuer Activity Code (IAC)——Online	5	Sets conditions for sending online transactions.	b
Issuer Country Code	2	Indicates country of issuer.	b

Note1: If the signatures for applications are not the same then the card must support SAD. For instance, if the card sets different CVM lists for domestic and international transactions and those lists contain signature data.

Note2: If the card has the ability to change signature data after it has been issued then that card must support SAD modification abilities.

Note3: If AIP also requires a signature, then SDA identifier lists must include AIP signatures. If DDA is supported, then it is recommended that AIP be used for signatures; there should be no other data labels aside from AIP.

10.8.2 Issuer CA-Issued Card Static Signatures

In accordance with Financial Integrated Circuit (IC) Card Debit/Credit Specifications, IC cards that support static data authentications must include the following data elements once they have undergone personalization:

- Authorization Centre Public Key Indices
- RID
- Issuer Public Key Certificates
- Signed Static Application Data: A variable-length data element generated by an issuer using the issuer private key associated with the issuer public/private key pair used by the issuer to authorize an issuer public key certificate; it is an electronic signature for important static data elements stored aboard an IC card.
- Issuer Public Key Reminders: a variable-length data element. Storage aboard an IC card is optional.
- Issuer Public Key Indicators: a variable-length data element supplied by an issuer.

Issuer CA must generate this data and transmit it to the data preparation system.

See Table 9 for an explanation of data objects used for static data authentication as required by Financial Integrated Circuit (IC) Card Debit/Credit Specifications; see Table 10 for card data used for static data authentication.

Table 9: Data Objects Used For Static Data Authentication

Label	Length	Value	Format
-	5	Registered application-supplied business identifier	b
'8F'	1	Authentication center public key index	b
'90'	N_{CA}	Issuer public key certificate	b

Label	Length	Value	Format
'92'	$N_I - N_{CA} + 36$	Issuer public key remainder (if present)	b
'9F32'	1 or 3	Issuer public key indicator	b
'93'	N_I	Signed static application data	b
-	Variable	Static data requiring authentication according to Section 9.3.1 of China Financial Integrated Circuit (IC) Card Specifications - Debit/Credit Card Specifications (See Table 21)	-

Table 10: Card Data Used For Static Data Authentication

Data Element	Description
CA Public Key Index (PKI)	Supplied to CA along with issuer public key certificate. Defines the CA public key used by the terminal to authenticate issuer public key certificates.
Issuer Public Key Certificate	Certificate includes issuer public keys signed by CA private keys
Issuer Public Key Indicator	Issuer public key certificate recovered using an RSA algorithm. Value is 3 or 65537
Issuer Public Key Remainder	Issuer public keys not included in the issuer public key certificate (if present)
Registered Application Identifier Aspect of AID	Used with CA public key index to identify terminal public keys

Data Element	Description
Signed Static Application Data (SAD)	A signature used to verify card static data. While a card is being personalized, SAD is written into the card after being signed by issuer private keys. It is recommended that the following data be used to generate the signature: AIP (if DDA is used). Application effective data, application expiry date, application main account number, application main account number sequence number, AUC, CVM list, issuer activity code-default, issuer activity code-refuse, issuer activity code-online, and issuer country code "(5F28)". If application signatures are not coherent then the card must support SAD. For instance, if the card sets different CVM lists for domestic and international transactions and those lists contain signature data.
SDA Label List	If AIP is also used for signatures then SDA label lists must include AIP labels. If DDA is supported then AIP should be used for signatures unless AIP cannot have other data labels.

Issuer CA-signed card static data is signed in accordance with the data shown in Table 11.

Table 11: Unsigned Static Application Data Formats

Field Name	Length	Description	Format
Recovered Data Header	1	Hexadecimal value '6A'	b
Signature Data Format	1	Hexadecimal value '03'	b
Hash Algorithm Identifier	1	Used to identify hash algorithm used to generate electronic signature hash values	b
Data Verification Code	2	Code distributed by issuer	b
Padding Bytes	$N_I - 26$	Padding bytes are values between $N_I - 26$ and composed of "BB" bytes	b

Field Name	Length	Description	Format
Hash Results	20	Hash values of static application data requiring authorization	b
Recovered Data End	1	Hexadecimal value 'BC'	b

10.8.2.1 Static Signature Data Composition

Issuer CA-signed static data is used to sign the data in Table 11; the hash results of Table 11 are results of running hash calculations on the sequentially-connected data of Table 8 (generated from the data of Table 10); the signature results of Table 7 data is signed card static data.

10.9 Card Personalization Procedures Keys and Cryptographic Algorithms

10.9.1 Card Personalization Procedure Security Regulations

The card personalization process includes the following data preparations and data personalization processes:

- IC card data required for personalized UICS debit/credit application cards is generated
- This data is transmitted to the card personalization system
- This data is then recorded into the card.

During the entire process the security of sensitive data must be protected such that it is not compromised or modified.

The issuer should guarantee the integrity of the following data during the card personalization process:

- Magnetic Strip Binary Data
- Cardholder Name
- Account Number PAN
- Account Number Sequence Number

- Card Expiry Date
- Relevant Public Key Certificates
- During the card personalization process, the issuer should also guarantee the secrecy and integrity of card risk management data, counter limits, and other forms of card data such as the following:
 - The card's three 3DES main keys
 - The card's RSA-based keys and signature data
- Cardholder PIN

Personalization data should be protected either by MAC or digital signatures. This protection should be effective from the moment the data leaves the generation system to storage within the card's memory. Personalization commands and load commands should be capable of processing MAC, signatures, or potentially encrypted data they are being transmitted to the card.

Cardholder PINs must be kept encrypted throughout the personalization process. Sensitive personalization data and keys must be encrypted while they are being transmitted and plaintext should only appear in hardware security modules or within the IC card. This data should be generated from within a hardware security module and be exported from said module in ciphertext form only. In the same way, the transmission keys used to calculate MAC and signatures and encrypted sensitive data should also only appear in ciphertext form and only be available in plaintext form either inside a hardware security module or IC card. Hardware security modules and IC card key encryption should occur in accordance with security guidelines.

For a given card, issuer public/private key pairs, IC card public/private key pairs, corresponding public key certificates, and stored IC card 3DES keys should all be generated within the same hardware security module. This data should only be exported from the hardware security module after it has been encryption.

10.9.2 Card Personalization Keys

Issuers should generate all card data during the card personalization process, including all of the Table 11 key data that must be written into the card; the issuer is also responsible for securely transmitting this information into the card. Issuer card personalization processes typically include data preparation systems and personalization systems; these systems may be internal to the issuer itself or

may belong to a contractor; in either case these systems must guarantee the integrity of all data at all times and as well as maintain the secrecy of sensitive card data such as PIN, keys, etc.

During the data preparation process, issuers should securely generate all of the key data and other UICS specified-card data listed in Table 11, securely transmit this data to the card personalization system, and then have the card personalization system securely write this data into the card. Card personalization process requires many different keys and therefore these keys must be generated before personalization can begin. Personalization process keys required by issuers and card data include:

- Issuer Main Key (KMC): Issuer main keys are generated by the issuer and shared with the data preparation system and personalization system. Issuer main keys are used to export card and card application keys K_{ENC} , K_{MAC} , and K_{DEK} .
- Card and card application ciphertext key K_{ENC} : used to generate card ciphertext and verify computer ciphertext.
- Card or card application MAC key K_{MAC} : used to lock the application area of China Financial Integrated Circuit (IC) cards as well as inspect personalization data being loaded onto cards during personalization to verify that data integrity is intact and no modifications have occurred.
- Card or card application encryption key K_{DEK} : Used to encrypt all sensitive data written into a card during personalization.

The key described above is used only for the protection of data used personalization.

Card encryption derivation key K_{ENC} is used to generate IC card ciphertext and verify main computer ciphertext. If the security level requires that STOREDATA command data segments are encrypted then this derivation key will also be used to decrypt these command data segments in CBC mode.

- K_{ENC} is a 16-byte (112-bit parity bit) 3DES key.

K_{ENC} is a key used for the following derivation:

$K_{ENC} : = 3DES(KMC)[6 \text{ lowest effective bytes of KEYDATA} \parallel 'F0' \parallel '01'] \parallel 3DES(KMC)[6 \text{ lowest effective bytes of KEYDATA} \parallel '0F' \parallel '01']$.

Card checksum derivation key K_{MAC} is used to check C-MAC used by EXTERNALAUTHENTICATE commands. If STOREDATA command ciphertext security guidelines require data to utilize MAC then this key is also used to check STORE DATA C-MAC.

- K_{MAC} is a 16-byte (112-bit parity bit) 3DES key.

K_{MAC} should be used for the following export methods:

$K_{ENC} = 3DES(K_{MC})[6 \text{ lowest effective bytes of KEYDATA} \parallel 'F0' \parallel '02'] \parallel 3DES(K_{MC})[6 \text{ lowest effective bytes of KEYDATA} \parallel '0F' \parallel '02']$.

Card key encryption derivation key K_{DEK} is used for the decryption of secret data received by STORE DATA commands under ECB mode.

- K_{DEK} is a 16-byte (112-bit parity bit) 3DES key.

K_{DEK} should be used for the following export methods:

$K_{DEK} = 3DES(K_{MC}) [6 \text{ lowest effective bytes of KEYDATA} \parallel 'F0' \parallel '03'] \parallel 3DES(K_{MC})[6 \text{ lowest effective bytes of KEYDATA} \parallel '0F' \parallel '03']$.

Additionally, issuer, data preparation systems and personalization systems should protect all personalization data being transmitted. The entire issuer personalization system also includes the following personalization keys:

- Key Exchange Key (KEK)----A 3DES key used for the encryption of issuer personalization input files as well as the protection of all data transmitted between issuers and data preparation systems; this key is shared by the issuer and the data preparation system. The KEK is unique for each issuer.
- Transport Key (DEK/TK)----A 3DES key used for encryption of sensitive issuer personalization output file data sent from the data preparation system to the personalization system. This key is shared between the data preparation system and personalization system.
- MAC Keys---A 3DES key used to protect the integrity of personalization files sent from the data preparation system to the personalization system.

All personalization keys are 16-byte 3DES keys; for specific usage and associated cryptographic computational models and algorithms, see the Personalization Guide and EMV General Personalization Specifications of UICS Specifications.

Issuer personalization systems are internal to institutions and may have different setups and deployments; additional keys may be required to protect data in certain places.

Table 12: Personalization Keys Summary

Name	Label	Type	Sharing Info	Usage	Generation
Issuer Main Keys	K _{MC}	3DES	Issuers, card companies, card merchants, personalization equipment	K _{MC} used to generate card keys (K _{ENC} , K _{MAC} , K _{DEC}), and write them into cards	Issuer
Card Data Encryption Keys	K _{ENC}	3DES	Issuers, card applications, card merchants, personalization equipment	Used to generate process keys to generate or decrypt ciphertext and CBC-mode encryptor data	Issuer
Card MAC Keys	K _{MAC}	3DES	Issuers, card applications, card merchants, personalization equipment	Used to generate process keys to generate or verify C-MAC required for command processes	Issuer
Card Data Encryption Keys	K _{DEC}	3DES	Issuers, card applications, card merchants, personalization equipment	Used to create communication keys for ECB-mode encryption of DES keys or the encryption of other kinds of sensitive data	Issuer
Issuer Key Exchange Keys	KEK _{ISS}	3DES	Issuers, data preparation system	Protects offline PIN and other sensitive data transmitted between issuer and data preparation equipment.	Issuer

Name	Label	Type	Sharing Info	Usage	Generation
Data Encryption Keys	DEK	3DES	Data preparation system, personalization system	Protects offline PIN and other sensitive data transmitted between data preparation equipment and personalization equipment.	Issuer
Transmission Keys	TK	3DES	Data preparation system, personalization system		Issuer
MAC Keys	MACkey	MAC	All personalization equipment	Used to protect personalization data files and supply personalization equipment with application data integrity.	Issuer

11 UICS Debit/Credit Application Transaction Cryptographic Operations

This chapter supplies actual key usage scenarios for UICS debit/credit applications. Including:

1. Online-Authorized Card Transactions:

- Card key export
- Export of UICS online transaction process keys
- ARQC calculations
- ARQC calculations
- TC calculations

2. PIN Changes:

- Export of security message encryption card keys
- Export of security message encryption process keys
- PIN lock encryption
- Export of card keys used for script MAC
- Export of process keys used for script MAC
- Script MAC calculations

3. Static Data Authentication:

- Verification of signed static application data

4. Dynamic Data Authentication:

- Verification of IC card public key certificates
- Signed dynamic data generated for DDA
- Verification of IC card public key certificates

- Verification of DDA signed dynamic data
- 5. Composite DDA/AC (CDA) Generation:
 - Offline counter device value encryption
 - CDA signed dynamic data generation
- 6. CDA Verification

Within this chapter, 3DES(K)[D] indicates a 3DES encryption calculation of key K on data block D. CBC_3DES(K)[D] indicates a 3DEA encryption calculation of key K on data block D in CBC mode. MAC(K)[M] indicates using utilizing ISO.IEC9797-1 calculation 3 on message M using key K and calculating an 8-byte MAC value using a DES derivation key.

11.1 Online-Authorized Card Transactions:

The section describes the usage of online-authorized card transaction encryption operations, including:

- Export of card main keys
- Export of procedure keys
- ARQC calculations
- ARPC calculations
- TC calculations

11.1.1 Export of Main Card Keys

Card AC main key MK_{AC} is created by using issuer AC main key IMK_{AC} to encrypt the connection of card PAN and PAN sequence numbers. See Section 9.1.1.

Here, issuer AC main key value IMK_{AC} is: 'F4 31 62 3E 49 97 89 C8 C2 F1 F7 25 AD 7F 89 49' with the connection of PAN and PAN sequence numbers as its input:

(The far-right 7 bytes of PAN (using an 'n' format, pad with '0' until 7-bytes).

When the connection of PAN and PAN sequence numbers are 8-bytes (16 numbers) long, then it can be used as input Y when written in numerical format; if the connection of PAN and PAN sequence numbers are less than 16 numbers, then align to the right and pad with '0' until it becomes an 8-byte numeral Y input; if the connection of PAN and PAN sequence numbers is more than 16 numbers, then Y= the furthest 16 numbers.

PAN value: '62 25 00 00 00 00 00 89'

PAN sequence number (1-byte, n coding).

PAN sequence number (1-byte, n coding) 1: '01'

Input is (Farthest right 16 numbers of PAN and PAN sequence number connection value):

'25 00 00 00 00 00 89 01'

For output: $MK_{AC} = 3DES(IMK_{AC})[input] || 3DES(IMK_{AC})[input \oplus 'FF FF FF FF FF FF FF FF'] =$

$3DES(IMK_{AC})['25 00 00 00 00 00 89 01'] || 3DES(IMK_{AC})['DA FF FF FF FF FF 76 FE'] =$

'89 E8 1E CA 27 6C 7D 50 A5 37 AE 51 2E 11 52 24'

Complete CHECKSUM value: '89 E9 1F CB 26 6D 7C 51 A4 37 AE 51 2F 10 52 25'

Therefore, $MK_{AC} = '89 E9 1F CB 26 6D 7C 51 A4 37 AE 51 2F 10 52 25'$.

11.1.2 Card AV Process Key Export

See Section 9.2 for information regarding card AC process key export.

Here, the card main key MK_{AC} value created using the calculation described above is the key:

'89 E9 1F CB 26 6D 7C 51 A4 37 AE 51 2F 10 52 25'

Input: Connection of ATC and six '0' bytes:

1. ATC value: '00 02'

2. Six '0' bytes: '00 00 00 00 00 00'
3. Connection: '00 02 00 00 00 00 00 00'

Output:

1. $ZL := 3DES(Key)['00' || '00' || '00' || '00' || '00' || '00' || ATC]$
2. $ZR := 3DES(Key)['00' || '00' || '00' || '00' || '00' || '00' || (ATC \oplus 'FFFF')]$
3. Calculation result: $ZL = '4F A1 BB 65 DC 84 58 BF'$, $ZR = 'ED 43 C2 10 F4 FD 47 11'$
4. $ZL || ZR = '4F A1 BB 65 DC 84 58 BF ED 43 C2 10 F4 FD 47 11'$

Complete post-CHECKSUM value:

'4F A1 BA 64 DC 85 58 BF EC 43 C2 10 F4 FD 46 10'

Therefore, card AC process key SK_{AC} is:

'4F A1 BA 64 DC 85 58 BF EC 43 C2 10 F4 FD 46 10'.

11.1.3 ARQC Generation

For ARQC generation, see Section 10.2.2.

The process key SK_{AC} used for ARQC generation is:

'4F A1 BA 64 DC 85 58 BF EC 43 C2 10 F4 FD 46 10'

Input is the connection of the following data:

1. Authorized amount: '00 00 00 01 68 00'
2. Other amount: '00 00 00 00 00 00'
3. Terminal country code: '01 56'
4. Terminal verification result: '00 00 00 80 00'
5. Transaction currency code: '01 56'

6. Transaction date: '07 11 09'
7. Transaction type: '01'
8. Unpredictable number: 'DC CD 7C 16'
9. AIP: '7D 00'
10. ATC: '00 02'
11. CVR: '03 A0 00 02'

The connection value is the input (if not a whole multiple of 8-bytes, then pad with '80' and as many '00' as necessary until it is):

'00 00 00 01 68 00 00 00 00 00 00 00 01 56 00 00 00 80 00 01 56 07 11 09 01
DC CD 7C 16 7D 00 00 02 03 A0 00 02 80 00 00'

Output:

$ARQC = MAC(SK_{AC})[input]:$

'8B 6F 6D 8F 49 64 A6 09'

11.1.4 ARPC Generation

For ARQC generation, see Section 10.2.2.

Here, process key SK_{AC} and the ARQC process key from above is the same and have the same process key export method. Its value is:

'4F A1 BA 64 DC 85 58 BF EC 43 C2 10 F4 FD 46 10'

Input is the connection of the following data:

1. $AC(ARQC)$: '8B 6F 6D 8F 49 64 A6 09'
2. ARC: '30 30'
3. Padded ARC: ARC padded with six '00's, creating an 8-byte string: '30 30 00 00 00 00 00 00'
4. Use XOR of ARQC and padded ARC to create the following input/output:

$$\text{ARPC} = 3\text{DES}(\text{SK}_{\text{AC}})[\text{input}] = 3\text{DES}(\text{SK}_{\text{AC}})[\text{AC} \oplus (\text{ARC} \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00' \parallel '00')]$$

'09 12 44 22 2A 16 3D FA'

11.1.5 TC Generation

For TC generation, see Section 10.2.2.

Here, process key SK_{AC} is the same as above:

'4F A1 BA 64 DC 85 58 BF EC 43 C2 10 F4 FD 46 10'

Input is the connection of the following data:

1. Authorized amount: '00 00 00 01 68 00'
2. Other amount: '00 00 00 00 00 00'
3. Terminal country code: '01 56'
4. Terminal verification result: '00 00 00 80 00'
5. Transaction currency code: '01 56'
6. Transaction date: '07 11 09'
7. Transaction type: '01'
8. Unpredictable number: 'DC CD 7C 16'
9. AIP: '7D 00'
10. ATC: '00 02'
11. CVR: '03 60 00 02'

The connection value is the input (if not a whole multiple of 8-bytes, then pad with '80' and as many '00' as necessary until it is):

'00 00 00 01 68 00 00 00 00 00 00 00 01 56 00 00 00 80 00 01 56 07 11 09 01
DC CD 7C 16 7D 00 00 02 03 60 00 02'

Output: $ARPC = MAC(SK_{AC})[Input] =$

'E4 65 D8 7C DD 8E 5A 48'

11.2 PIN Changes

This section describes completing the encryption operation by issuers of PIN changes within secure messages, including:

- Export of card secure message encryption main keys
- Export of secure message encryption process keys
- PIN lock encryption
- Export main card keys calculated by script MAC
- Script MAC process key export
- Script MAC calculations

11.2.1 Script Encryption Card Main Key Export

For script encryption card main key export, see Section 10.1

Key: issuer secure message encryption main key IMK_{SMC} with a value of:

'76 5E EC AB B0 A8 61 08 F1 CB 15 91 73 B9 7F 26'

Input is the connection of the following data (PAN and PAN sequence number):

When the connection of PAN and PAN sequence numbers are 8-bytes (16 numbers) long, then it can be used as input Y when written in numerical format; if the connection of PAN and PAN sequence numbers are less than 16 numbers, then align to the right and pad with '0' until it becomes an 8-byte numeral Y input; if the connection of PAN and PAN sequence numbers is more than 16 numbers, then Y= the furthest 16 numbers.

For PAN 6225000000000089: '62 25 00 00 00 00 00 89'

PAN sequence number (1-byte, n coding).

PAN sequence number (1-byte, n coding) 1: '01'

Input: '25 00 00 00 00 00 89 01'

Output should follow: $MK_{SMC} = 3DES(IMK_{SMC})[input] || 3DES(IMK_{SMC})[input \oplus 'FF FF FF FF FF FF FF'] = 3DES(IMK_{SMC})['25 00 00 00 00 00 89 01'] || 3DES(IMK_{SMC})['DA FF FF FF FF FF 76 FE'] =$:

'26 CC 64 66 C9 E2 0D 5A 04 5A 18 51 68 04 F9 1F'

Complete post-CHECKSUM value:

'26 CD 64 67 C8 E3 0D 5B 04 5B 19 51 68 04 F8 1F'

Script encryption card key MK_{SMC} is:

'26 CD 64 67 C8 E3 0D 5B 04 5B 19 51 68 04 F8 1F'

11.2.2 Script Encryption Process Key Export

For script encryption process key export, see Section 10.3.1.

See the section above for the export of the card main key MK_{SMC} , used here for secure message encryption;

'26 CD 64 67 C8 E3 0D 5B 04 5B 19 51 68 04 F8 1F'

Input:

1. ATC: '00 02'
2. Process key A generated input: '00 00 00 00 00 00' || ATC
3. Process key B generated input: '00 00 00 00 00 00' || (ATC $\oplus$ 'FFFF')

Output:

Process key A: $= Z_L = 3DES(Key)['00' || '00' || '00' || '00' || '00' || '00' || ATC] = '3D BC 12 FE 5F D5 09 0B'$

Process key B: $= Z_R = 3DES(Key)['00' || '00' || '00' || '00' || '00' || '00' || (ATC \oplus 'FFFF')] = 'BF 05 36 C2 83 76 A4 DF'$

The connection of process key A and process key B is:

'3D BC 12 FE 5F D5 09 0B BF 05 36 C2 83 76 A4 DF'

Complete post-CHECKSUM value:

'3D BC 13 FE 5E D5 08 0B BF 04 37 C2 83 76 A4 DF'

Therefore, the script encryption process key SK_{SMC} is:

'3D BC 13 FE 5E D5 08 0B BF 04 37 C2 83 76 A4 DF'

11.2.3 Script Encryption

For secure message encryption, see Section 10.3.2.

The secure message encryption key SK_{SMC} is the same as the above section:

'3D BC 13 FE 5E D5 08 0B BF 04 37 C2 83 76 A4 DF'

Input:

1. The data that must be encrypted is: '04 12 34 FF 37 1C F2 A4'
2. Prepare data source according to encryption format to act as input data: '08 04 12 34 FF 37 1C F2 A4'

Output:

Input data is encrypted using ciphertext process encryption key SK_{SMC} using an ECB-mode 3DES. Result:

'64 52 95 FC F6 6D 07 5B 21 48 F1 21 79 0B E0 CC'

11.2.4 Script MAC Card Main Key Export

For secure message MAC calculations, see Section 10.3.2.

Here, the secure message integrity issuer main key is IMK_{SMI} . Assuming that the IMK_{SMI} value is:

'AB AB F7 C8 FB 7C 40 3D 8F F7 3E A8 D0 F1 CE 38'

Input is the connection of the following data (PAN and PAN sequence number):

When the connection of PAN and PAN sequence numbers are 8-bytes (16 numbers) long, then it can be used as input Y when written in numerical format; if the connection of PAN and PAN sequence numbers are less than 16 numbers, then align to the right and pad with '0' until it becomes an 8-byte numeral Y input; if the connection of PAN and PAN sequence numbers is more than 16 numbers, then Y= the furthest 16 numbers.

For PAN 6225000000000089: '62 25 00 00 00 00 00 89'

PAN sequence number (1-byte, n coding).

PAN sequence number (1-byte, n coding) 1: '01'

Input: '25 00 00 00 00 00 89 01'

For output =, follow $MK_{SMI} = 3DES(IMK_{SMI})[input] || 3DES(IMK_{SMI})[input \oplus 'FF FF FF FF FF FF FF'] =$

$3DES(IMK_{SMI})['25 00 00 00 00 00 89 01'] || 3DES(IMK_{SMI})['DA FF FF FF FF FF 76 FE'] =$:

'C5 B4 C9 C2 46 A0 A8 C4 49 77 57 0C 13 02 CB 89'

Complete post-CHECKSUM value:

'C4 B5 C8 C2 46 A1 A8 C4 49 76 57 0D 13 02 CB 89'

Therefore, card script MAC main key MK_{SMI} is:

'C4 B5 C8 C2 46 A1 A8 C4 49 76 57 0D 13 02 CB 89'

11.2.5 Script MAC Process Key

For secure message (script) MAC process key export, see Section 10.3.1.

Here, the secure message integrity (script MAC) card main key MK_{SMI} is the same as above and is:

'C4 B5 C8 C2 46 A1 A8 C4 49 76 57 0D 13 02 CB 89'

Input:

1. ATC: '00 02'

2. Process key A generated input: '00 00 00 00 00 00' || ATC
3. Process key B generated input: '00 00 00 00 00 00' || (ATC $\oplus$ 'FFFF')

Output:

Process key A := $Z_L = 3DES(MK_{SMI})['00' || '00' || '00' || '00' || '00' || '00' || ATC]$ =

'3D BC 12 FE 5F D5 09 0B'

Process key B := $Z_R = 3DES(MK_{SMI})['00' || '00' || '00' || '00' || '00' || '00' || (ATC \oplus 'FFFF')]$ =

'BF 05 36 C2 83 76 A4 DF'

The connection of process key A and process key B is:

'3D BC 12 FE 5F D5 09 0B BF 05 36 C2 83 76 A4 DF'

Complete post-CHECKSUM value:

'3D BC 13 FE 5E D5 08 0B BF 04 37 C2 83 76 A4 DF'

Therefore, script MAC process key SK_{SMI} is:

'3D BC 13 FE 5E D5 08 0B BF 04 37 C2 83 76 A4 DF'

11.2.6 Pin Change Script MAC Calculations

For secure message (script) MAC calculations, see Section 10.3.2.

Here, the secure message integrity process key SK_{SMI} is the same as above and is:

'3D BC 13 FE 5E D5 08 0B BF 04 37 C2 83 76 A4 DF'

Input:

Input data used to calculate MAC = command header || ATC || AC || data domain (optional) || 80...

MAC input data is the following:

'84 24 00 02 14 00 02 8B 6F 6D 8F 49 64 A6 09 64 52 95 FC F6 6D 07 5B 21
48 F1 21 79 0B E0 CC 80'

The MAC process key calculation result using the MAC algorithm is:

'E2 4D 5F F1 45 CD 65 B5'

Select first 4 bytes for MAC value: 'E2 4D 5F F1'

Final script command:

'84 24 00 02 14 64 52 95 FC F6 6D 07 5B 21 48 F1 21 79 0B E0 CC E2 4D 5F
F1'

11.3 Static Data Authentication

This section describes encryption operations conducted by terminals for card static authorization processes.

The steps required for terminal verification of signed static application data includes:

1. Use IC card Root CA public key to verify issuer public key certificate
2. Use the step described above to obtain signed static application verified by issuer public key
3. Obtain data authorization code

This section describes the second certification calculation for signed static application data.

11.3.1 Verification of Signed Static Application Data

11.3.1.1 Issuer Public Keys

Issuer public keys can be extracted directly from issuer public key certificates and issuer public key remainders. For issuer public key certificate format and content, see Financial IC Card Debit/Credit Application Root CA Public Key Authentication Specifications-Technical Specifications and Section 10.7 of these standards.

Issuer Public Key Modulus:

C2 AB E7 63 CD 75 D5 7D DC D3 4C F6 32 AA 27 F5 E9 5A 52 04 56 2C 2D
 39 E9 46 07 74 C7 61 B8 65 73 E9 D4 C1 B5 AC 4D AD A9 F4 2F 92 17 71 2B
 73 D5 A6 6E 29 EA 8E 02 74 08 5F F6 33 CB 8E BB FA FB 13 F8 BC 82 63
 84 E1 52 2F AB 4F C4 54 58 18 CB 6F 41 65 85 84 5E 7E 64 B7 21 A3 4B E4
 8F AE F0 B0 78 DC BA DE BE 5F FA 22 A7 47 FF AB C8 EC F6 2F E4 B0 96
 94 9F AE 88 A3 31 79 28 73 16 3B EC D9 0D 75 D8 F1 57 0F 47 ED 40 F7 86
 90 B7 FB

Issuer Public Key Indicator: '03'

Signed static data (Tag '93') as follows:

A9 F0 A9 59 A1 FC FC 7D 49 E0 40 8D BF D5 5B 8D A8 E0 E1 94 AB C8 79
 E6 21 52 8F 70 29 43 4D 70 FB B8 06 B6 65 9E 77 EE C2 0C 52 71 EE 6C EB
 96 9D 05 47 F2 AA 70 80 10 AD DF 6A 0E 59 AD 60 1A 3A 2F 26 3D 0F F5
 92 54 80 52 CF 37 2D 94 58 CC 9C A8 D7 54 1F 93 A0 A6 E7 95 BB EB 25
 9B 1C F3 20 94 2B 10 59 93 65 50 1F EC 5E 3A 2E D9 99 C8 1D FE E4 3E 97
 85 34 FC 82 85 EC 5C A4 32 FD E4 F5 23 49 BC C5 54 FE CC F6 5E AC 66
 C7 B4 7D 72

Execute the following verification process:

1. Inspect and determine whether signed static application data length and issuer public key modulus length is the same (digits/bytes): compare results: identical (1152/144)
 2. Use RSA algorithm to recover signed data source text
- Issuer public key indicator=03, then: $X = (\text{signed static application data})^{03} \bmod (\text{issuer public key modulus value})$ X is recovered data.
 - Recover original data from signed static application data like as follows:

6A 03 01 DA C1 BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
 BB
 BB
 BB
 BB
 BB
 BB BB 4B 47 C5 8D 86 67 C3 10 44 40 AD DC F4 7C D9 8D 89 37 B2 CB BC

3. Inspect the accuracy of the format of the recovered data:

- Data header (first byte): '6A', inspection result: pass.
 - Format (second byte): '03', inspection result: pass.
 - Data end (last byte): 'BC', inspection result: pass.
4. Connect data used for hash calculation and recover data required for hash calculation (second to fifth data source; i.e. all data except header, end, and hash result data):

03 01 DA C1 BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB

AFL explains what authenticated static application data is required for hash calculations; the data is as follows (data sequence is not changeable):

Application effective date: '5F 25 03 07 01 01'

Application expiry date: '5F 24 03 12 12 31'

Account number (PAN): '5A 08 62 25 00 00 00 00 89'

Account number sequence number: '5F 34 01 01'

IAC—default: '9F 0D 05 FC F0 E4 08 00'

IAC—refuse: '9F 0E 05 00 10 00 00 00'

IAC—online: '9F 0F 05 FC F8 E4 F8 80'

Application usage control: '9F 07 02 FF C0'

Cardholder verification method list: '8E 12 00 00 00 00 00 00 00 5E 03 41
03 42 03 60 03 1F 00'

SDA label list: '9F 4A 01 82'

AIP: '7D 00'

3. Verification of IC card public key certificate and extraction of IC card public key

4. Verification of signed dynamic application data

This section describes the steps described above but does not include verification of the issuer public key certificate. Verification of the issuer public key certificate is similar to the verification of IC card public key certificates.

11.4.1 Generation of Dynamic Application Data Signature

IC Card private key (standard form):

Modulus:

9E B6 F2 D3 47 2F 73 35 B9 9F 03 D8 CD 69 E7 06 B7 E6 5B 8F 5A DD CE
0E 9A 00 9D C8 16 42 C3 F9 99 D3 C5 92 82 D0 4D B8 42 FC 68 58 09 B5 A7
99 76 88 AD 75 E2 C6 9D 91 18 97 27 E9 EF F0 A5 8D FE F8 31 37 D6 A9 15
9F 81 9E 31 61 0A C9 78 9A F1 33 BA 0A 4B 92 BD 1A 92 B9 E2 9E 6C 8B
62 01 67 03 E7 83 AA F8 57 D3 C4 BA 29 9B D3 CA C8 57 48 78 51 54 07 54
08 37 BB FC 6D F2 AE B5 6B 19 1F FF DF 36 6E CA D6 5A EB 15 10 77 1E
5E E5 8D

Private key indicator (d):

69 CF 4C 8C DA 1F A2 23 D1 14 AD 3B 33 9B EF 59 CF EE E7 B4 E7 3E 89
5F 11 55 BE 85 64 2C 82 A6 66 8D 2E 61 AC 8A DE 7A D7 52 F0 3A B1 23
C5 10 F9 B0 73 A3 EC 84 69 0B 65 BA 1A 9B F5 4B 19 09 54 A5 76 25 39 C6
0E 69 49 01 1D 81 C5 C2 B3 7A 43 56 18 92 4C FE 89 DE 8F 24 B5 56 B9 65
60 8D 49 64 A5 AF 9E 8B 3E AD ED BF 59 9F 18 9A F5 DA A6 00 03 43 91
79 CB 24 77 6A D4 90 D4 59 10 9E 1D 28 DC DB EB 0D 6B 20 B8 A4 03 5A
FB 84 AA 93

RSA private key (CRT Mode):

Prime p:

D8 0A 38 E8 40 1A E7 3A 58 64 78 D9 A5 AA CB 3E 26 7D 73 6A 5E 07 E8
19 A2 9C 92 88 B3 C2 36 21 BF 73 96 E8 89 3B 6E A6 69 4F A6 71 A1 5B 2E
2C 3A 40 52 27 8A 48 D0 BD 6B 52 12 E0 E6 D8 E6 C2 F3 73 F9 AC 5C 5F
0A 63

Prime q:

BC 12 4C 36 22 0A 84 29 33 CE 1C 55 32 6A 23 0E 95 85 5F 31 F8 6B 69 13
D6 50 5C 73 89 65 43 AD 20 A7 8C 44 A5 A6 E8 E8 E6 28 A5 FD 0B C2 29
54 CE 9B DC F1 E5 E7 01 6E 88 F0 81 0B A7 5D CE E6 E2 AB 11 C2 48 B8
DB 4F

dp:

90 06 D0 9A D5 67 44 D1 90 42 FB 3B C3 C7 32 29 6E FE 4C F1 94 05 45 66
6C 68 61 B0 77 D6 CE C1 2A 4D 0F 45 B0 D2 49 C4 46 35 19 A1 16 3C C9 72
D1 80 36 C5 06 DB 35 D3 9C E1 61 EB 44 90 99 D7 4C F7 FB C8 3D 94 B1
97

dq:

7D 61 88 24 16 B1 AD 70 CD 34 12 E3 76 F1 6C B4 63 AE 3F 76 A5 9C F0 B7
E4 35 92 F7 B0 EE 2D 1E 15 C5 08 2D C3 C4 9B 45 EE C5 C3 FE 07 D6 7D
61 88 24 16 B1 AD 70 CD 34 12 E3 76 F1 6C B4 63 AE 3F 76 A5 9C F0 B7 E4
35 92 F7 B0 EE 2D 1E 15 C5 08 2D C3 C4 9B 45 EE C5 C3 FE 07 D6 C6 38
89 BD 3D F6 99 44 AB 9F 05 F5 AB 5D 1A 3E 89 EF 41 C7 61 2C 30 7B 3C
DF

Qinv:

46 29 3F FE 04 43 C7 17 F6 88 EE 6D 03 78 0F B8 42 9B 2A AF 5D EB 2B FE
7E 56 DA 0A 5C 34 02 AA 09 19 6A 01 8B 30 F1 0C 2C 7C AF B7 30 1C 64
6B FE 20 16 54 CA DE 09 DF E4 AC 98 B8 B0 B7 D5 9D 71 AE 33 34 D1 D2
98 83

RSA Public Key:

Module:

9E B6 F2 D3 47 2F 73 35 B9 9F 03 D8 CD 69 E7 06 B7 E6 5B 8F 5A DD CE
0E 9A 00 9D C8 16 42 C3 F9 99 D3 C5 92 82 D0 4D B8 42 FC 68 58 09 B5 A7
99 76 88 AD 75 E2 C6 9D 91 18 97 27 E9 EF F0 A5 8D FE F8 31 37 D6 A9 15
9F 81 9E 31 61 0A C9 78 9A F1 33 BA 0A 4B 92 BD 1A 92 B9 E2 9E 6C 8B
62 01 67 03 E7 83 AA F8 57 D3 C4 BA 29 9B D3 CA C8 57 48 78 51 54 07 54
08 37 BB FC 6D F2 AE B5 6B 19 1F FF DF 36 6E CA D6 5A EB 15 10 77 1E
5E E5 8D

Public key indicator: '03'

Connect the input data for hash calculation in the following order:

Signed data format: '05'

Hash algorithm indication: '01' (SHA-1)

Card dynamic data length: '03'

Card dynamic application data: '02 00 02'

Pad (144-28=116 bytes, IC card public modulus length is 144 bytes) until the result is as follows:

05 01 03 02 00 02 BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB
BB
BB
BB
BB
BB BB

The unpredictable number must be connected to the end of the data described above:

'57 15 45 79'

All dynamic application data used for calculating hash values are as follows:

05 01 03 02 00 02 BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB
BB
BB
BB
BB
57 15 45 79

Use SHA-1 algorithm to compute hash value of the above data:

'20 68 CE 12 C5 37 75 E4 44 54 B6 68 EF 76 9D 91 0E D1 00 CD'

Generate signature:

First, connect data using the following sequence:

Data header: '6A'

73 D5 A6 6E 29 EA 8E 02 74 08 5F F6 33 CB 8E BB FA FB 13 F8 BC 82 63
84 E1 52 2F AB 4F C4 54 58 18 CB 6F 41 65 85 84 5E 7E 64 B7 21 A3 4B E4
8F AE F0 B0 78 DC BA DE BE 5F FA 22 A7 47 FF AB C8 EC F6 2F E4 B0 96
94 9F AE 88 A3 31 79 28 73 16 3B EC D9 0D 75 D8 F1 57 0F 47 ED 40 F7 86
90 B7 FB

Issuer public key indicator: '03'

IC card public key certificate (Tag '9F46') is as follows:

30 C4 90 DB A9 3C ED D0 04 97 34 B8 97 39 CD C7 61 39 11 F9 44 B2 4B
D0 72 FC 87 5E B5 A1 FF 52 E0 F1 90 FE 3C F7 74 3F D1 2F CA F7 57 ED
DC E7 A3 A1 93 60 AB 19 BF 9E 0F 1D B7 6C E9 91 BA 7E 57 9E 49 D6 8C
F2 A6 E2 4C A4 9A 36 75 D5 F4 7D 4A F9 FA 30 8A A4 21 0C A6 DF 9A 42
0C 0F E9 D2 CC E8 33 39 EC C9 53 7E B2 D9 35 48 5D BB DE 2D 82 28 8E
DE 62 F0 A9 2B E0 17 5C 89 99 52 E2 82 FC 2D 6B F1 DF 48 CB B9 26 99 F7
3E 01 EF CB 31

Execute the following verification process to verify IC card public key certificate:

1. Inspect whether or not IC card public key certificate length is the same as issuer public key modulus length: compare results: identical, 1152 digits or 144 bytes.
2. Use RSA algorithm to recover signed data source text:

Issuer public key indicator=03, then: $X = (\text{IC card public key certificate})_3 \bmod (\text{issuer public key modulus number})$, X is recovered data.

The original data recovered from IC public key certificate is as follows:

6A 04 62 25 00 00 00 00 89 FF FF 12 20 00 00 02 01 01 90 01 9E B6 F2 D3
47 2F 73 35 B9 9F 03 D8 CD 69 E7 06 B7 E6 5B 8F 5A DD CE 0E 9A 00 9D
C8 16 42 C3 F9 99 D3 C5 92 82 D0 4D B8 42 FC 68 58 09 B5 A7 99 76 88 AD
75 E2 C6 9D 91 18 97 27 E9 EF F0 A5 8D FE F8 31 37 D6 A9 15 9F 81 9E 31
61 0A C9 78 9A F1 33 BA 0A 4B 92 BD 1A 92 B9 E2 9E 6C 8B 62 01 67 03
E7 83 AA F8 82 44 00 08 41 1F 61 55 12 63 DF D8 60 40 57 65 D4 1E 8A 6F
BC

3. Inspect the accuracy of the format of the recovered data:

Recovered data header (first byte): '6A'.

Inspection result: accurate.

Certificate format (second byte): '04'.

Inspection result: accurate.

Recovered data end (last byte): 'BC'.

Inspection result: accurate.

4. Connect data to be used to calculate hash values:

Recover data required for hash calculation (second to fifth data source; i.e. all data except header, end, and hash result data):

```
04 62 25 00 00 00 00 89 FF FF 12 20 00 00 02 01 01 90 01 9E B6 F2 D3 47
2F 73 35 B9 9F 03 D8 CD 69 E7 06 B7 E6 5B 8F 5A DD CE 0E 9A 00 9D C8
16 42 C3 F9 99 D3 C5 92 82 D0 4D B8 42 FC 68 58 09 B5 A7 99 76 88 AD 75
E2 C6 9D 91 18 97 27 E9 EF F0 A5 8D FE F8 31 37 D6 A9 15 9F 81 9E 31 61
0A C9 78 9A F1 33 BA 0A 4B 92 BD 1A 92 B9 E2 9E 6C 8B 62 01 67 03 E7
83 AA F8
```

IC card public key and public key remainder required for hash value calculation:

Issuer public key indicator: '03'

IC card public key remainder:

```
57 D3 C4 BA 29 9B D3 CA C8 57 48 78 51 54 07 54 08 37 BB FC 6D F2 AE
B5 6B 19 1F FF DF 36 6E CA D6 5A EB 15 10 77 1E 5E E5 8D
```

AFL explains what authenticated static application data is required for hash calculations; the data is as follows (data sequence is not changeable):

Application effective date: '5F 25 03 07 01 01'

Application expiry date: '5F 24 03 12 12 31'

Account number (PAN): '5A 08 62 25 00 00 00 00 89'

Account number sequence number: '5F 34 01 01'

IAC—default: '9F 0D 05 FC F0 E4 08 00'

IAC—refuse: ‘9F 0E 05 00 10 00 00 00’

IAC—online: ‘9F 0F 05 FC F8 E4 F8 80’

Application usage control: '9F 07 02 FF C0'

Cardholder verification method list:

```
'8E 12 00 00 00 00 00 00 00 5E 03 41 03 42 03 60 03 1F 00'
```

SDA label list: '9F 4A 01 82'

AIP: '7D 00'

The connection of static data requiring authentication is as follows:

```
5F 25 03 07 01 01 5F 24 03 12 12 31 5A 08 62 25 00 00 00 00 89 5F 34 01
01 9F 0D 05 FC F0 E4 08 00 9F 0E 05 00 10 00 00 00 9F 0F 05 FC F8 E4 F8 80
9F 07 02 FF C0 8E 12 00 00 00 00 00 00 00 5E 03 41 03 42 03 60 03 1F 00
9F 4A 01 82 7D 00
```

All data used to calculate hash is as follows:

```
04 62 25 00 00 00 00 00 89 FF FF 12 20 00 00 02 01 01 90 01 9E B6 F2 D3 47
2F 73 35 B9 9F 03 D8 CD 69 E7 06 B7 E6 5B 8F 5A DD CE 0E 9A 00 9D C8
16 42 C3 F9 99 D3 C5 92 82 D0 4D B8 42 FC 68 58 09 B5 A7 99 76 88 AD 75
E2 C6 9D 91 18 97 27 E9 EF F0 A5 8D FE F8 31 37 D6 A9 15 9F 81 9E 31 61
0A C9 78 9A F1 33 BA 0A 4B 92 BD 1A 92 B9 E2 9E 6C 8B 62 01 67 03 E7
83 AA F8 57 D3 C4 BA 29 9B D3 CA C8 57 48 78 51 54 07 54 08 37 BB FC
6D F2 AE B5 6B 19 1F FF DF 36 6E CA D6 5A EB 15 10 77 1E 5E E5 8D 03
5F 25 03 07 01 01 5F 24 03 12 12 31 5A 08 62 25 00 00 00 00 00 89 5F 34 01
01 9F 0D 05 FC F0 E4 08 00 9F 0E 05 00 10 00 00 00 9F 0F 05 FC F8 E4 F8 80
9F 07 02 FF C0 8E 12 00 00 00 00 00 00 00 00 5E 03 41 03 42 03 60 03 1F 00
9F 4A 01 82 7D 00
```

5. The hash calculation identifier (second byte of hash input data described above, '01') indicates the usage of hash function SHA-1; result of the SHA-1 function on the above data is as follows:

```
'82 44 00 08 41 1F 61 55 12 63 DF D8 60 40 57 65 D4 1E 8A 6F'
```

- Obtain hash value from recovered data as follows:

‘82 44 00 08 41 1F 61 55 12 63 DF D8 60 40 57 65 D4 1E 8A 6F‘

7. Compare hash values: identical.
8. Inspect whether PAN recovered from IC card certificate is the same as card PAN.

IC card PAN: ‘62 25 00 00 00 00 00 89’

PAN recovered from certificate: ‘62 25 00 00 00 00 00 89 FF FF’

PAN comparison result: identical.

9. Inspect whether or not the IC card certificate is still within its effective date range; inspect whether the last day of the month listed on the expiry date is equal to or after the current date. If it is not, then IC card public key certificate verification fails.

IC card certificate effective date (MM YY): ‘12 20’. Or (YY YYMM): ‘20 20 12’

Current date (YY YY MM): ‘20 07 11’

Certificate expiration inspection result: pass.

10. Inspect recovered data and determine whether or not the IC card public key label can be identified

If IC card public key algorithm label is ‘01’, then the label can be identified as RSA. IC card public key certificate verification is successful.

From X above, connect the left-hand side of the IC card public key modulus and the IC card public key modulus remainder to obtain the IC card public key modulus below:

9E B6 F2 D3 47 2F 73 35 B9 9F 03 D8 CD 69 E7 06 B7 E6 5B 8F 5A DD CE
0E 9A 00 9D C8 16 42 C3 F9 99 D3 C5 92 82 D0 4D B8 42 FC 68 58 09 B5 A7
99 76 88 AD 75 E2 C6 9D 91 18 97 27 E9 EF F0 A5 8D FE F8 31 37 D6 A9 15
9F 81 9E 31 61 0A C9 78 9A F1 33 BA 0A 4B 92 BD 1A 92 B9 E2 9E 6C 8B
62 01 67 03 E7 83 AA F8 57 D3 C4 BA 29 9B D3 CA C8 57 48 78 51 54 07 54
08 37 BB FC 6D F2 AE B5 6B 19 1F FF DF 36 6E CA D6 5A EB 15 10 77 1E
5E E5 8D

11.4.3 Signed Dynamic Application Data Verification

IC card public key:

Obtain IC card public key modulus from IC card public key certificate:

```
9E B6 F2 D3 47 2F 73 35 B9 9F 03 D8 CD 69 E7 06 B7 E6 5B 8F 5A DD CE
0E 9A 00 9D C8 16 42 C3 F9 99 D3 C5 92 82 D0 4D B8 42 FC 68 58 09 B5 A7
99 76 88 AD 75 E2 C6 9D 91 18 97 27 E9 EF F0 A5 8D FE F8 31 37 D6 A9 15
9F 81 9E 31 61 0A C9 78 9A F1 33 BA 0A 4B 92 BD 1A 92 B9 E2 9E 6C 8B
62 01 67 03 E7 83 AA F8 57 D3 C4 BA 29 9B D3 CA C8 57 48 78 51 54 07 54
08 37 BB FC 6D F2 AE B5 6B 19 1F FF DF 36 6E CA D6 5A EB 15 10 77 1E
5E E5 8D
```

IC card public key indicator: '03'

Signed dynamic application data:

```
47 5A F0 B1 72 48 D6 CD 3A D5 35 3F C5 8C 46 42 F4 AB FD 02 FA F5 95
14 E8 77 DF CF F6 FF 5B 01 93 6B A5 36 BA 14 0B A8 69 3B FD 8A 6C 76
EE 4A 22 F7 9D 39 3F DC B9 AE 81 D6 E8 E0 EE 12 98 2F 36 38 8B 04 B7 03
CF 18 B3 91 E0 93 20 58 D8 BD 91 02 95 04 E8 8C BF F2 7C 80 28 44 DC E7
96 DC FF 06 26 7A 32 97 D5 73 81 C5 BC F1 B9 B9 44 9D AB 25 F0 87 62 69
E3 EA 5C 4F 17 48 62 A3 B3 9F C1 31 F5 FD 7B 7F 03 B9 2A 8A 6E AC 92
F7 B9 1D
```

Signature verification occurs with the following steps:

1. Inspect signed dynamic application data length and determine if IC card public key indicator is identical to IC card public key modulus length

Inspection result: identical, 1152 digits or 144 bytes.

2. Use RSA algorithm and IC public key recovered signed dynamic application data with the following result:

```
6A 05 01 03 02 00 02 BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB 20 68 CE 12 C5 37 75 E4 44 54 B6 68 EF 76 9D 91 0E D1 00 CD BC
```


3. Inspect accuracy of recovered data format:

Recovered data header (first byte): '6A'.

Inspection result: Pass.

Recovered data end (last byte): 'BC'.

Inspection result: pass

4. Connect data to be used to calculate hash values:

Recover data required for hash calculation (second to tenth data source; i.e. all data except header, end, and hash result data):

05 01 03 02 00 02 BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB
BB
BB
BB
BB BB

Connect unpredictable number used for hash calculation to above data: '57 15 45 79'.

All data to be used for the hash calculation is as follows:

[illegible]

5. The hash calculation identifier (second byte of hash input data described above, '01') indicates the usage of hash function SHA-1. The hash result of the SHA-1 function described above is as follows:

```
'20 68 CE 12 C5 37 75 E4 44 54 B6 68 EF 76 9D 91 0E D1 00 CD'
```

6. Obtain hash value from recovered data as follows:

'20 68 CE 12 C5 37 75 E4 44 54 B6 68 EF 76 9D 91 0E D1 00 CD'

7. Compare hash values: identical. Verification of signed dynamic application data successful.

11.5 CDA Generation and Verification

11.5.1 Composite DDA/application ciphertext generation (CDA)

This section supplies examples of composite DDA/AC generation; application ciphertext is TC.

Generation of signed dynamic application data within CDA:

Step One: Factor Generate AC command input data and GPO command input data

GenerateAV command input data (CDOL1 associated data):

Authorized amount: '00 00 00 00 01 00'

Other amount: '00 00 00 00 00 00'

Terminal country code: '01 56'

Transaction currency code: '01 56'

Transaction date: '07 11 09'

Transaction type: '01'

Unpredictable number: 'BB 31 E4 A4'

GPO command input data: '01 56'

Step two: Compute signed application data to generate transaction data hash value.

Input data for transaction hash value includes: GPO command input data, GenAC command input data, and other GenerateAC response data aside from SDAD.

GPO command input data: '01 56'

GenAC command data:

'00 00 00 00 01 00 00 00 00 00 00 00 01 56 00 00 00 00 01 56 07 11 09 01
BB 31 E4 A4'

Other GenerateAC response data aside from SDAD:

'9F 27 01 40 9F 36 02 00 01 9F 10 08 07 01 0A 03 90 00 02 01'

All data used to calculate hash is as follows:

01 56 00 00 00 00 01 00 00 00 00 00 00 00 01 56 00 00 00 00 01 56 07 11 09
01 BB 31 E4 A4 9F 27 01 40 9F 36 02 00 01 9F 10 08 07 01 0A 03 90 00 02 01

Use SHA-1 algorithm to compute hash value of the above data:

'3E 93 51 D2 21 BC DF 2F 69 44 0C 4E 3A C6 8B E8 31 93 4B 49'

Step Three: Calculate hash value of unsigned dynamic application data

Connect unsigned dynamic application data in the following order:

Signed data format: '05'

Hash algorithm identifier: '01'

Card dynamic data length: '20'

IC card dynamic number: '02 00 01'

Application ciphertext type (CID): '40'

Application ciphertext (AC): 'B4 AD F3 53 F2 21 E0 54'

Transaction data hash value:

'3E 93 51 D2 21 BC DF 2F 69 44 0C 4E 3A C6 8B E8 31 93 4B 49'

Pad (144-57=87bytes, here IC card modulus length is 144 bytes):

Unpredictable number (terminal random number): 'BB 31 E4 A4'

All dynamic application data used for calculating hash values are as follows:

```
05 01 20 02 00 01 40 B4 AD F3 53 F2 21 E0 54 3E 93 51 D2 21 BC DF 2F 69
44 0C 4E 3A C6 8B E8 31 93 4B 49 BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
```

Use SHA-1 algorithm to compute hash value of the above data:

```
'7D 32 6A 62 A0 1E 47 42 C8 E8 42 E9 EA 0A 17 40 BC 74 59 04'
```

Step Four: Use IC card private key to sign dynamic application data

Connect all data in the following sequence:

1. Data header: '6A'
2. Compute far-left 144-22=122 bytes of hash value computed from dynamic application data (i.e. unsigned dynamic application data):

05 01 20 02 00 01 40 B4 AD F3 53 F2 21 E0 54 3E 93 51 D2 21 BC DF 2F 69
44 0C 4E 3A C6 8B E8 31 93 4B 49 BB BB BB BB BB BB BB BB BB BB BB
BB
BB
BB
BB BB

- ### 3. Dynamic Application Data Hash Values:

```
'7D 32 6A 62 A0 1E 47 42 C8 E8 42 E9 EA 0A 17 40 BC 74 59 04'
```

4. Data end: 'BC'

Utilize the data above to obtain the following result:

```
6A 05 01 20 02 00 01 40 B4 AD F3 53 F2 21 E0 54 3E 93 51 D2 21 BC DF 2F
69 44 0C 4E 3A C6 8B E8 31 93 4B 49 BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB BB
A0 1E 47 42 C8 E8 42 E9 EA 0A 17 40 BC 74 59 04 BC
```

Use RSA algorithm and IC card public key on data above to calculate signed dynamic application data; signed application data = $(\text{input})^d \bmod (\text{IC card public key modulus})$

92 E3 1D 96 B3 72 CE EA 7E CC C7 A5 96 AF 02 A0 2E E3 7C 6B 2B C2 F8
B8 07 2A 9A FB 39 03 1D D6 2F 4F 39 45 1A 07 0E C7 51 FC F9 36 02 04 07
7E A6 E9 61 72 1B E6 20 36 26 9F B2 FB 57 E2 4E A4 3B 09 05 E2 43 17 1D
7C 8C 6E F8 32 85 39 A0 E7 EB 0F B2 1B 1D DD E8 7E FB 6E F6 BE 03 D5
3F 0E 38 A3 79 1B EB 18 61 7B F6 C0 71 F8 02 BB 3E FB 49 E6 A8 06 27 50
D4 AE F9 8A F2 73 4C 66 D0 73 1E 9F EF 1B 69 4F D3 70 5A 61 7C 66 19 12
7D 79

This completes the generation of CDA signed data.

GenAC response data should be as follows:

77 81 A8 9F 27 01 40 9F 36 02 00 01 9F 4B 81 90 92 E3 1D 96 B3 72 CE EA
7E CC C7 A5 96 AF 02 A0 2E E3 7C 6B 2B C2 F8 B8 07 2A 9A FB 39 03 1D
D6 2F 4F 39 45 1A 07 0E C7 51 FC F9 36 02 04 07 7E A6 E9 61 72 1B E6 20
36 26 9F B2 FB 57 E2 4E A4 3B 09 05 E2 43 17 1D 7C 8C 6E F8 32 85 39 A0
E7 EB 0F B2 1B 1D DD E8 7E FB 6E F6 BE 03 D5 3F 0E 38 A3 79 1B EB 18
61 7B F6 C0 71 F8 02 BB 3E FB 49 E6 A8 06 27 50 D4 AE F9 8A F2 73 4C 66
D0 73 1E 9F EF 1B 69 4F D3 70 5A 61 7C 66 19 12 7D 79 9F 10 08 07 01 0A
03 90 00 02 01

11.5.2 CDA Verification

Step One: Factor GenerateAC command input data and GPO command input data

1. GenAC command input data:

Authorized amount: '00 00 00 00 01 00'

Other amount: '00 00 00 00 00 00'

Terminal country code: '01 56'

Terminal verification result: '00 00 00 00 00'

Transaction currency code: '01 56'

Transaction type: '01'

Unpredictable number: 'BB 31 E4 A4'

2. GenAC command response data:

[9F27] CID: '40'

[9F36] ATC: '00 01'

3. [9F4B] signature dynamic application data:

92 E3 1D 96 B3 72 CE EA 7E CC C7 A5 96 AF 02 A0 2E E3 7C 6B 2B C2 F8
B8 07 2A 9A FB 39 03 1D D6 2F 4F 39 45 1A 07 0E C7 51 FC F9 36 02 04 07
7E A6 E9 61 72 1B E6 20 36 26 9F B2 FB 57 E2 4E A4 3B 09 05 E2 43 17 1D
7C 8C 6E F8 32 85 39 A0 E7 EB 0F B2 1B 1D DD E8 7E FB 6E F6 BE 03 D5
3F 0E 38 A3 79 1B EB 18 61 7B F6 C0 71 F8 02 BB 3E FB 49 E6 A8 06 27 50
D4 AE F9 8A F2 73 4C 66 D0 73 1E 9F EF 1B 69 4F D3 70 5A 61 7C 66 19 12
7D 79

4. [9F10] IAD: '07 01 0A 03 90 00 02 01'

5. GPO command input data: '01 56'

Step two: Verify accuracy of signed dynamic application data

IC Card Public modulus:

9E B6 F2 D3 47 2F 73 35 B9 9F 03 D8 CD 69 E7 06 B7 E6 5B 8F 5A DD CE
0E 9A 00 9D C8 16 42 C3 F9 99 D3 C5 92 82 D0 4D B8 42 FC 68 58 09 B5 A7
99 76 88 AD 75 E2 C6 9D 91 18 97 27 E9 EF F0 A5 8D FE F8 31 37 D6 A9 15
9F 81 9E 31 61 0A C9 78 9A F1 33 BA 0A 4B 92 BD 1A 92 B9 E2 9E 6C 8B
62 01 67 03 E7 83 AA F8 57 D3 C4 BA 29 9B D3 CA C8 57 48 78 51 54 07 54
08 37 BB FC 6D F2 AE B5 6B 19 1F FF DF 36 6E CA D6 5A EB 15 10 77 1E
5E E5 8D

Issuer public key indicator: '03'

Signed dynamic application data (Tag9F4B):

92 E3 1D 96 B3 72 CE EA 7E CC C7 A5 96 AF 02 A0 2E E3 7C 6B 2B C2 F8
B8 07 2A 9A FB 39 03 1D D6 2F 4F 39 45 1A 07 0E C7 51 FC F9 36 02 04 07
7E A6 E9 61 72 1B E6 20 36 26 9F B2 FB 57 E2 4E A4 3B 09 05 E2 43 17 1D
7C 8C 6E F8 32 85 39 A0 E7 EB 0F B2 1B 1D DD E8 7E FB 6E F6 BE 03 D5
3F 0E 38 A3 79 1B EB 18 61 7B F6 C0 71 F8 02 BB 3E FB 49 E6 A8 06 27 50

05 01 20 02 00 01 40 B4 AD F3 53 F2 21 E0 54 3E 93 51 D2 21 BC DF 2F 69
 44 0C 4E 3A C6 8B E8 31 93 4B 49 BB BB BB BB BB BB BB BB BB BB BB
 BB
 BB
 BB
 BB
 BB

Connect unpredictable number used for hash calculation to above data 'BB 31 E4 A4'

All data to be used for the hash calculation is as follows:

05 01 20 02 00 01 40 B4 AD F3 53 F2 21 E0 54 3E 93 51 D2 21 BC DF 2F 69
 44 0C 4E 3A C6 8B E8 31 93 4B 49 BB BB BB BB BB BB BB BB BB BB BB
 BB
 BB
 BB
 BB
 BB
 BB

5. The hash calculation identifier (second byte of hash input data described above, '01') indicates the usage of hash function SHA-1. The hash result of the SHA-1 function described above is as follows:

'7D 32 6A 62 A0 1E 47 42 C8 E8 42 E9 EA 0A 17 40 BC 74 59 04'

6. Obtain hash value from recovered data as follows:

'7D 32 6A 62 A0 1E 47 42 C8 E8 42 E9 EA 0A 17 40 BC 74 59 04'

7. Compare hash values: identical.

Verification of signed dynamic application data successful.

Step three: Verify accuracy of transaction data hash values

Input data for calculating transaction hash values include: GPO command input data, and other GenAC command input data aside from signed dynamic application data.

GPO command input data: '01 56'

GenAC command input data:

'00 00 00 00 01 00 00 00 00 00 00 00 01 56 00 00 00 00 01 56 07 11 09 01

BB 31 E4 A4'

Other GenerateAC response data aside from signed dynamic application data:

'9F 27 01 40 9F 36 02 00 01 9F 10 08 07 01 0A 03 90 00 02 01'

Connection value of above data (hash function input):

'01 56 00 00 00 00 01 00 00 00 00 00 00 01 56 00 00 00 00 01 56 07 11
09 01 BB 31 E4 A4 9F 27 01 40 9F 36 02 00 01 9F 10 08 07 01 0A 03 90 00 02
01'

Recovered transaction hash values from signed dynamic application data is as follows:

'3E 93 51 D2 21 BC DF 2F 69 44 0C 4E 3A C6 8B E8 31 93 4B 49'

Recalculate transaction hash values as follows:

'3E 93 51 D2 21 BC DF 2F 69 44 0C 4E 3A C6 8B E8 31 93 4B 49'

Verification result: hash values identical, CDA verification successful.